The Role of Entanglement in Solving Differential Equations

THESIS

submitted in partial fulfillment of the
requirements for the degree of

MASTER OF SCIENCE
in
QUANTUM INFORMATION SCIENCE & TECHNOLOGY

Author :	Roman Ilic
Student ID :	s4116488 | 6104134
Supervisors :	Vedran Dunjko
	Riccardo Molteni
Second correctors :	Evert van Nieuwenburg
	Matthias Möller

Leiden, The Netherlands, June 20, 2025

The Role of Entanglement in Solving Differential Equations

Roman Ilic

Instituut-Lorentz, Leiden University
P.O. Box 9506, NL-2300 RA Leiden, The Netherlands

June 20, 2025

Abstract

Solving differential equations (DEs) is fundamental across scientific and engineering domains, yet classical numerical methods face severe limitations when applied to high-dimensional, stiff, or nonlinear systems. In recent years, quantum computing has been proposed as a potential solution to some of these problems, and one particular paradigm——Differentiable Quantum Circuits (DQCs)——has emerged as a promising framework for learning DE solutions by leveraging the expressivity of parameterized quantum models. However, the extent to which quintessentially quantum features—such as entanglement—contribute to this potential advantage remains unclear.

This thesis investigates when and how entanglement becomes a necessary resource for DQCs to solve DEs effectively. Three research questions guide the study: (1) How expressive and trainable are DQCs in approximating solution functions? (2) To what extent can classical Matrix Product States (MPS), with maximal bond dimension, replicate DQC results? (3) What is the minimum entanglement (quantified via MPS bond dimension) required to accurately approximate a given DE solution? We address these questions across three representative DE problems: a damped harmonic oscillator, a strongly coupled system of oscillators, and a stationary instance of the Navier–Stokes equations. DQCs are implemented with Chebyshev-based quantum feature maps supporting automatic differentiation; MPS serve as classical benchmarks with tunable entanglement capacity.

Results show that DQCs achieve high-quality approximations in low to moderately stiff regimes, but their performance deteriorates with increasing stiffness and coupling, highlighting limitations in trainability under

shallow depth. MPS with sufficient bond dimension can reproduce DQC results up to numerical precision, but diverge in optimization dynamics due to differing model architectures. Crucially, entanglement-constrained MPS experiments reveal specific bond dimension thresholds below which solution quality significantly degrades, thereby quantifying the minimum entanglement required for faithful representation. In particular, nonlinear contributions in systems such as Navier–Stokes cannot be captured without nontrivial entanglement.

Overall, this work establishes a direct connection between entanglement and function approximation capability in quantum models for differential equations. The findings provide insight into which structural features of DEs—such as stiffness, nonlinearity, and variable coupling—–intrinsically demand entanglement, thereby outlining the regimes in which DEs would benefit most from quantum.

Contents

List of Figures

Chapter 1

Introduction

1.1 Background

Differential equations (DEs) serve as the foundational framework for describing the evolution of systems in physics, chemistry, biology, energy, and finance. Whether modeling the fluid dynamics of atmospheric flows, the diffusion of chemical species, or the pricing of financial products, DEs define functional relationships between variables and their derivatives subject to initial and boundary conditions. In theory, exact solutions exist only for a narrow class of low-dimensional linear systems. Most real-world problems require numerical solutions [31]. A significant bottleneck in solving DEs numerically arises from the curse of dimensionality. For a system involving d continuous variables discretized into M points each, the resulting function space—–the space of all candidate functions mapping those discretized inputs to outputs, from which an approximate solution is selected—–scales as $\mathcal{O}(M^d)$. This exponential increase in memory and computational requirements severely limits the scalability of traditional methods. Many DEs of practical interest are further complicated by structural features that exacerbate this challenge: nonlinearity, stiffness, correlations, chaos, and many others. Stiffness where equations involve components evolving on widely separated timescales (e.g., chemical kinetics), forcing solvers to adopt very small time steps to maintain stability[1] and accuracy. Strong correlations where variables are tightly coupled (e.g., multi-asset financial models), leading to dense, ill-conditioned sys-

[1]Numerical instability refers to the amplification of small computational errors—like round-off or truncation—into large deviations from the true solution.

tems that are difficult to decouple and solve efficiently. Chaotic behavior where small perturbations in initial conditions grow exponentially over time (e.g., turbulent fluid flows), making long-term numerical integration unstable and highly sensitive to round-off or discretization errors. These features, combined with high dimensionality, define a class of problems that often exceed the reach of traditional numerical methods [14][41][25]. Classical solvers are divided into local and global strategies. Local methods discretize the variable space and approximate derivatives using numerical differentiation schemes, such as finite-differencing or Runge-Kutta integration [6]. These approaches are straightforward to implement and perform well for smooth, low-dimensional problems. However, in high-dimensional settings, they require fine grids, leading to large linear systems. Furthermore, the use of approximated derivatives introduces truncation errors – cumulative numerical approximation errors that can degrade stability and accuracy over time. Global methods, such as spectral methods, assume that the solution can be approximated as a weighted sum of known basis functions (e.g., Chebyshev or Fourier series - [38]). The task then becomes finding the optimal set of coefficients for this expansion that satisfies the differential equation. These methods are known to exhibit exponential convergence for sufficiently smooth problems, as the approximation error decays exponentially with the number of basis functions used [5]. However, they generate large, dense system matrices that must be solved globally. This global coupling of degrees of freedom can be a major disadvantage in problems with localized features, non-smooth solutions, or irregular boundary conditions, where the entire system must be adjusted even for local changes, leading to poor scalability and high computational cost. Quantum algorithms have been proposed to accelerate certain classes of linear problems. The well-known algorithm by Harrow, Hassidim, and Lloyd (HHL) targets the solution of linear systems of the form $A\vec{u} = \vec{b}$, where A is a discretized version of the differential operator [15]. To represent the boundary condition $\vec{b}$, the algorithm typically employs *amplitude encoding*[2], in which the entries of a classical vector are mapped onto the amplitudes of a quantum state, providing an exponential memory advantage. However, preparing an arbitrary state $|b\rangle$ generally requires circuits of exponential depth[3] (a.k.a. the *input problem*). The *output problem* arises when reading out the function values from the solution state $|u\rangle \approx A^{-1}|b\rangle$, which generally requires exponential sampling. Moreover, HHL-based solvers rely on finite-difference discretiza-

[2] *Amplitude encoding* stores a vector of 2^N amplitudes using N qubits only.

[3] Note: exponential depth in the number of qubits N.

tion of the differential operator and, therefore, inherit the same scaling limitations for high-dimensional or complex DEs.
To better accommodate nonlinear, high-dimensional, or data-sparse scenarios, machine learning methods such as Physics-Informed Neural Networks (PINNs) have been proposed [33]. PINNs incorporate the governing DEs directly into the loss function of a deep neural network, allowing the model to learn solution functions by minimizing the residual of the DE using automatic differentiation (AD). When measurement data is available, an additional term can be included in the loss to encourage agreement with observations. This approach eliminates the need for mesh-based discretization and is naturally suited to problems with sparse data, uncertain initial conditions, or inverse modeling requirements. It is also capable of tackling stiff and nonlinear systems by leveraging the flexibility of deep neural networks. However, these advantages come with a considerable training cost: optimization typically requires tens of thousands of function and gradient evaluations. They also suffer from instability due to rugged loss landscapes, sensitivity to initialization, and generalization to complex solution spaces [44]. Quantum machine learning seeks to exploit the exponentially large Hilbert space[4] of quantum systems to expand the range of function classes accessible to learning models. A central architecture in this context is the Parameterized Quantum Circuit (PQC), also referred to as a Quantum Neural Network (QNN). PQCs are universal function approximators whose expressivity, in the sense of A. Manzano et al. [26], refers to their ability to approximate any function within a particular function class and domain up to arbitrary precision, with respect to a chosen distance metric. This expressivity depends on the interplay of several architectural components. The quantum feature map plays a foundational role by defining the function space within the Hilbert space — it determines the basis set of functions accessible to the circuit [36]. The variational ansatz (the structure and parameterization of quantum gates) and the entangling architecture (how qubits interact) govern the circuit's capacity to explore and transform that space. The circuit depth and parameter count directly impact how finely the PQC can approximate target functions within the accessible set. Finally, the measurement scheme (choice of observables) determines how the function values are projected and extracted, effectively filtering the output space. Together, these components define the function space accessible to the PQC and shape its capacity to approximate target functions. This distinction is crucial in sci-

[4] A Hilbert space is a complete vector space with an inner product, generalizing Euclidean space to possibly infinite dimensions.

entific applications such as solving differential equations: performance depends not only on how well the true solution aligns with the PQC's expressible function space, but more fundamentally, on whether the function lies within its closure under a suitable norm — that is, whether the circuit can approximate the solution to a desired level of accuracy, even if it cannot represent it exactly. A hybrid quantum-classical optimization loop is used to iteratively adjust the parameters of the variational circuit, refining the measured projection to approximate the target function. This setup allows PQCs to function effectively within expressive function spaces while remaining compatible with NISQ[5] hardware: shallow circuits limit decoherence, and the classical optimizer absorbs most of the computational burden, mitigating the impact of gate noise and hardware imperfections [21]. Crucially, quantum circuit evaluations are separable — each function value or derivative corresponds to an independent circuit execution enabling scalable, parallel evaluations that align well with the architectural constraints of NISQ-era devices. Differentiable Quantum Circuits (DQCs) extend the PQC framework by introducing input-dependent feature maps that are explicitly designed to allow analytical differentiation with respect to the input variables. In practice, this means that the derivatives of the target function can be computed by applying differentiation rules directly to the quantum feature map. As a result, both the function values and their derivatives are represented as expectation values of observables, allowing differential constraints to be embedded into the loss function. This approach mirrors the philosophy of classical PINNs, but is implemented within a quantum latent space. The feature maps used in DQCs are often built from nonlinear transformations (e.g., Chebyshev or Fourier-inspired encodings [45]), which allow for the representation of highly oscillatory, stiff, or more complex behaviors. As with other variational quantum algorithms, DQCs face training challenges, including barren plateaus and local minima [28]. Despite these difficulties, their high expressivity, compatibility with automatic differentiation, and structural fit with differential equation-solving workflows make DQCs a promising tool for tackling the combined challenges of high dimensionality, nonlinearity, and chaotic systems in scientific computing.

Matrix Product States (MPS) offer a structured and efficient representation of quantum states, particularly advantageous for systems exhibiting limited entanglement [42][34]. Unlike PQCs and DQCs, which define parameterized circuits, MPS models directly represent the quantum state as a

[5]Noisy Intermediate-Scale Quantum devices with up to few hundred qubits and non-corrected noise.

sequence of low-rank tensors connected via bond dimensions. These bond dimensions dictate the maximum entanglement entropy that can be captured between subsystems, providing precise control over the complexity of the state space. Notably, when the bond dimension is sufficiently large to encompass the entanglement generated by a quantum circuit, MPS can simulate the circuit's output exactly, ensuring that, given the same input, both representations yield identical outputs, up to numerical precision. However, the internal parameterizations differ, potentially leading to variations in optimization trajectories and convergence behaviors. This distinction underscores the importance of understanding the structural nuances between different quantum representations. Moreover, theoretical bounds on entanglement entropy elucidate the regimes where quantum circuits can achieve exponential speedups over classical approximators: when a problem's entanglement surpasses the capacity of an MPS with polynomial bond dimension, classical simulation becomes infeasible [11]. Consequently, MPS models serve as both practical tools for benchmarking quantum circuits and theoretical frameworks for delineating the boundaries of quantum advantage.

1.2 Objectives

This study investigates the role of entanglement in the efficacy of differentiable quantum models for solving differential equations. The central question is: **What is the role of quintessentially quantum features in the efficacy of these models?**
To address this, this study sets out three main objectives:

1. **Evaluate the expressivity and learnability of Differentiable Quantum Circuits (DQCs).** This involves replicating key results from foundational DQC studies, with a focus on how accurately these models approximate solutions within their defined function spaces and under relevant distance metric:

   How effective are DQCs in terms of expressivity and learnability?

2. **Benchmark DQC performance against Matrix Product States (MPS).** This objective assesses whether an MPS-based simulation of a given DQC can reproduce its results when using a sufficiently large bond dimension. The goal is to evaluate whether such simulations can match the approximation quality of the original quantum circuit, and

to analyze differences in convergence behavior:

> *To what extent can an MPS-based simulation recover DQC results with maximal bond dimension?*

3. **Characterize the entanglement requirements of differential equation solutions.** This involves systematically reducing the bond dimension of the MPS and monitoring solution quality to identify thresholds below which accuracy degrades. The objective is to determine the minimum amount of entanglement necessary for faithful representation:

> *What is the minimum bond dimension required to accurately capture the solution of a given differential equation?*

Beyond these core objectives, the study explores broader questions regarding the functional role of entanglement in circuit expressivity, as formalized in the framework of A. Manzano et al. [26]. Specifically:

- How does entanglement influence the quality and granularity of function approximation?
- Which structural features of a target solution become inaccessible as entanglement is reduced?
- In what scenarios do differential equations intrinsically demand high entanglement for accurate representation?

Together, these investigations aim to clarify not only whether entanglement enhances quantum model performance, but under what conditions it becomes essential.

1.3 Report Structure

Chapter 1 introduces the motivation, context, and challenges of solving differential equations using quantum machine learning, and defines the three core research objectives: evaluating the expressivity and trainability of Differentiable Quantum Circuits (DQCs), comparing their performance to Matrix Product States (MPS) with maximal bond dimension, and identifying the minimum entanglement required for accurate approximation. Chapter 2 provides the theoretical foundation, presenting DQCs as parameterized quantum circuits with a focus on their workflow, trainability, and generalization properties. It then introduces the MPS formalism, emphasizing the role of entanglement entropy and bond dimension truncation, and concludes with a review of related studies to contextualize the novelty of this work. Chapter 3 outlines the methodology used to address each research objective, detailing the experimental setup and models applied to three representative differential equations. Chapter 4 presents the results, structured by research question and example. Chapter 5 synthesizes the insights gained across all experiments to answer the research questions, examine limitations, and reflect on open challenges. Chapter 6 concludes the thesis by summarizing the main findings, revisiting unresolved issues, and proposing directions for future research. The latter also includes a reflection on the impact and interdisciplinary positioning of this thesis. The thesis concludes with a link to the code repository, followed by acknowledgments, references, and an appendix containing relevant mathematical derivations and a summary of the computational resources used.

Chapter 2

Theory

2.1 DQCs

The idea of DQCs has been developed by A. Paine et al. [18] where both target functions and their derivatives are approximated by PQCs. Throughout the paper we will refer to the DQC as the one approximating the functions to make the text more fluent, although technically it is the expectation value of the DQC that approximates the function. The parametrized feature maps allow for automatic differentiation via nonlinear mappings. They benefit from a rich basis of functions (e.g., Chebyshev polynomials) which contributes to DQCs high expressivity in approximating complex function classes with relatively shallow depth.

2.1.1 General Workflow

The general optimization workflow of DQCs is illustrated in Fig. 2.1. First, we specify the input for the solver including a set of differential equations with their respective boundary conditions. Then, we choose the composition of the quantum circuit such as a feature map, a variational ansatz with certain depth, a cost operator (i.e. an observable), a loss function, and finally a strategy to match boundary terms and derivatives. Note we will use *floating boundary hanlding* here which forces the model to meet boundary conditions at every training step ($f(0) = u(0)$ where $u(0)$ corresponds to the target function evaluated at 0). This guarantees exact matching to initial values but does not remove the dependence on the initial variational parameters θ_{init} (i.e. the initial parametrization could still influence the search for θ_{opt}). Next, we need to specify the classical optimizer with associated hyperparameters such as the learning rate, including the number of iterations or epochs and other potential exit conditions. A specific optimization schedule could also be chosen. All elements and strategies contained in Fig. 2.1 can be found in A. Paine et al.'s work [18]. We will only consider a subset in this study.

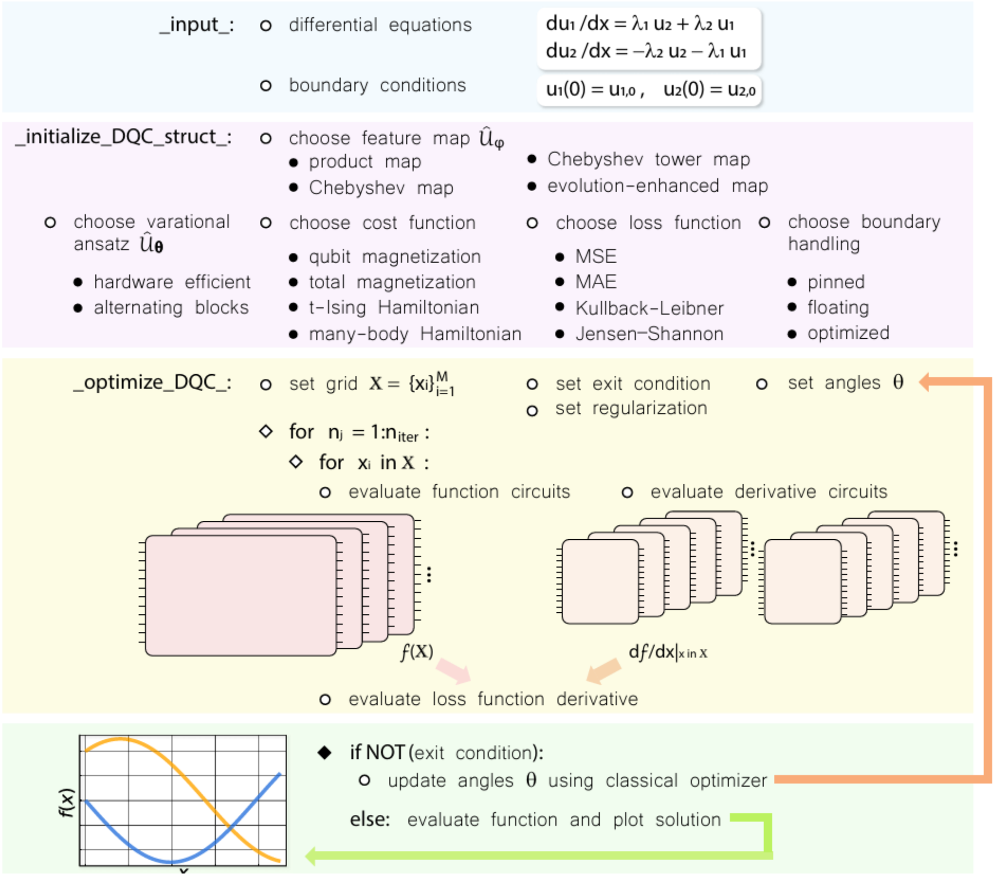

Figure 2.1: DQCs Optimization Workflow. *The problem is modeled as a system of differential equations with functions u, variables x, and boundary conditions. A derivative quantum circuit is built using a chosen feature map, ansatz, and cost operator. The loss, based on a selected distance metric, measures the gap between predicted and reference solutions. The circuit is evaluated on a grid of points to compute function and derivative values, which are used to calculate the loss gradient. A hybrid quantum-classical loop then updates the variational parameters using an optimizer until convergence. Figure reproduced from [18].*

Once the DQC structure and optimization schedule are defined, we must specify a set of points for each equation variable (e.g., Uniform grid, Chebyshev grid, or randomly-drawn grid). The expectation value over a variational quantum state for the cost operator $\hat{C}$ is estimated for the chosen point x: $f(x) = \langle f_{\varphi,\theta}(x)|\hat{C}|f_{\varphi,\theta}(x)\rangle$ where $|f_{\varphi,\theta}(x)\rangle = \hat{\mathcal{U}}_\theta \hat{\mathcal{U}}_{\varphi(x)}|\varnothing\rangle$. Here, $|\varnothing\rangle$ denotes the initial quantum register of N qubits all initialized in the computational basis state $|0\rangle^{\otimes N}$. This serves as the standard starting point for most quantum circuits. The input x is embedded into this state via the feature map unitary $\hat{U}_{\varphi(x)}$, which encodes classical information into quantum phases. The x-dependent amplitudes are sculptured to represent a potential solution at θ_{init}, alleviating the *input problem* with *amplitude encoding* for preparing the solution at boundary [15]. The *output problem* is also being avoided as the high dimensional function representation is projected to its scalar values by measuring the expectation of an observable. Then, derivative quantum circuits are constructed and their expectation value is estimated using the *parameter shift rule*, avoiding the accumulated error from approximate derivatives characteristic to numerical differentiation:

$$\frac{df(x)}{dx} = \frac{1}{2}\sum_j \left(\langle f^{+}_{d\varphi,j,\theta}(x)|\hat{C}|f^{+}_{d\varphi,j,\theta}(x)\rangle - \langle f^{-}_{d\varphi,j,\theta}(x)|\hat{C}|f^{-}_{d\varphi,j,\theta}(x)\rangle\right)$$

where $|f^{\pm}_{d\varphi,j,\theta}(x)\rangle$ is obtained through the parameter shifting of the j individual quantum operations used in the feature map $\hat{\mathcal{U}}_{\varphi(x)}$. Note the use of the *parameter shift rule* is restricted to the case of quantum feature maps generated by Pauli matrices or any involutory matrix. For me details, see O. Kyriienko et al.'s work on differentiation rules of quantum circuits [17]. We repeat the procedure for all training points and compose the loss function for the entire grid. The loss function assigns a score to the potential solution satisfying the differential equation(s), matching both derivative terms and function polynomials to minimize the loss:

$$\theta_{\text{opt}} = \arg\min_\theta \mathcal{L}_\theta(d_x f, f, x)$$

Note that the optimizer searching for a suitable solution in the exponential space of fitting functions resembles the spectral method with improved scaling and supports parallel training [28]. Using a gradient-based procedure, we compute the gradient of the loss function with respect to θ. We repeat all previous steps until we reach the exit condition. Finally, we extract the full solution on testing points by sampling the cost function.

2.1.2 Ingredients

M. Schuld et al. showed that PQCs naturally represent Fourier-type sums and, in this sense, can become universal approximators for any square-integrable function defined on $[0, 2\pi]^N$ under the L^2 distance [36]. A. Manzano et al. extended the approximability of PQCs to the space of continuous functions, the space of p-integrable functions, and the H^k Sobolev space with a simple min-max feature scaling [26]. PQCs can therefore be written as a generalized trigonometric series in the following way:

$$f_\theta(x) = \langle 0|U^\dagger(x;\theta) M U(x;\theta)|0\rangle = \sum_{\omega\in\Omega} c_\omega(\theta) e^{i\omega x} \tag{2.1}$$

where $x \in \Re$ is linearly encoded by a Hamiltonian e^{ixH} (e.g., Pauli gate) whose eigenvalues constitute the frequency spectrum of the model $w = \lambda_i - \lambda_j \in \Omega$ — the accessible functions defined by frequencies of a single encoding layer composed of single-qubit gates generated by the same Pauli operator with eigenvalues λ_i and λ_j. For integer-valued frequencies, we end up with a partial Fourier series of orthogonal basis functions where $\Omega \in \mathbb{Z}^N$. The coefficients $c_\omega(\theta)$ on the other hand are controlled by the rest of the circuit via θ. The representable function class depends therefore on the feature x, the variational parameter θ, and the model itself. M. Schuld et al. also claims that with r repeated layers of the quantum feature map interleaved with variational blocks, the frequency spectrum Ω contains r frequencies. This means that for a single encoding layer ($r = 1$), the associated PQC will only be able to represent a simple sine function (a single-frequency Fourier series). However, DQCs encode x nonlinearly in the exponential Hilbert space which significantly increases the set of accessible functions despite relying on single layers of single-qubit encoding gates. We will consider the following feature maps introduced by A. Paine et al.: the *Chebyshev feature map* and the *Chebyshev tower feature map*. The **Chebyshev feature map** belongs to the product feature map family and uses a single-qubit rotation $\hat{R}_{y,j}(\varphi(x))$ for each qubit, with nonlinearity introduced as $\varphi(x) = 2\arccos(x)$:

$$\hat{U}_\varphi(x) = \bigotimes_{j=1}^{N} \hat{R}_{y,j}(2 \cdot \arccos(x)) \tag{2.2}$$

This nonlinear feature map introduces homogeneous Chebyshev polynomials of degree-1. The **Chebyshev tower feature map** is simply the standard Chebyshev feature map with $\varphi(x) = 2j \cdot arcosx$ which introduces higher-degree Chebyshev polynomials with no increase in system size or number of rotations:

$$\hat{\mathcal{U}}_{\varphi}(x) = \bigotimes_{j=1}^{N} \hat{R}_{y,j}(2j \cdot \arccos(x)) \tag{2.3}$$

where the expansion of the rotation using Euler's formula gives degree-j Chebyshev polynomials of the first and second kind denoted as $T_j(x)$ and $U_j(x)$:

$$\begin{aligned}
\hat{R}_{y,j}(\varphi(x)) &= \exp\left(-i\frac{2j \cdot \arccos(x)}{2}\hat{Y}_j\right) \\
&= \cos(j \arccos(x))\, \mathbb{1}_j - i \sin(j \arccos(x))\, \hat{Y}_j \\
&= T_j(x)\, \mathbb{1}_j + \sqrt{1-x^2}\, U_{j-1}(x)\, \hat{X}_j \hat{Z}_j
\end{aligned}$$

The Chebyshev expansion can be seen as a Fourier-like series with a non-linear transformation of x through $arcos(x)$. The Chebyshev basis is therefore a powerful basis set of fitting polynomials that enables rich expressivity needed for highly nonlinear functions.
The derivative circuits can be obtained by applying the *parameter shift rule* to the parametrized encoding gates. We derive the cost function of the derivative circuit for a general product feature map:

$$\begin{aligned}
\frac{d}{dx}\hat{\mathcal{U}}_{\varphi}(x) &= \frac{1}{2}\left(\frac{d}{dx}\varphi(x)\right) \sum_{j'=1}^{N} \bigotimes_{j=1}^{N} (-i\hat{Y}_{j'}\delta_{j,j'})\hat{R}_{y,j}(\varphi(x)) \\
&= \frac{1}{2}\left(\frac{d}{dx}\varphi(x)\right) \sum_{j'=1}^{N} \bigotimes_{j=1}^{N} \hat{R}_{y,j}(\varphi(x) + \pi\delta_{j,j'})
\end{aligned}$$

where x-dependent rotations are shifted one at a time and where the cost function can be further obtained using the *chain rule* as the following:

$$\frac{d}{dx}\langle\varnothing|\hat{\mathcal{U}}^{\dagger}_{\varphi(x)}\hat{C}\hat{\mathcal{U}}_{\varphi(x)}|\varnothing\rangle = \frac{1}{4}\left(\frac{d}{dx}\varphi(x)\right)\left(\langle\hat{C}\rangle^{+} - \langle\hat{C}\rangle^{-}\right) \tag{2.4}$$

where $\langle\hat{C}^{+}\rangle$ and $\langle\hat{C}^{-}\rangle$ are the sums of shifted unitaries:

$$\langle\hat{C}^{\pm}\rangle = \textstyle\sum_{j'=1}^{N} \bigotimes_{j=1}^{N} \langle\varnothing|\hat{R}^{\dagger}_{y,j}(\varphi(x) \pm \frac{\pi}{2}\delta_{j,j'})\hat{C}\hat{R}_{y,j}(\varphi(x) \pm \frac{\pi}{2}\delta_{j,j'})|\varnothing\rangle$$

M. Schuld et al. defines circuit expressivity as the *flexibility* of the Fourier coefficients to explore and transform the space of accessible basis functions

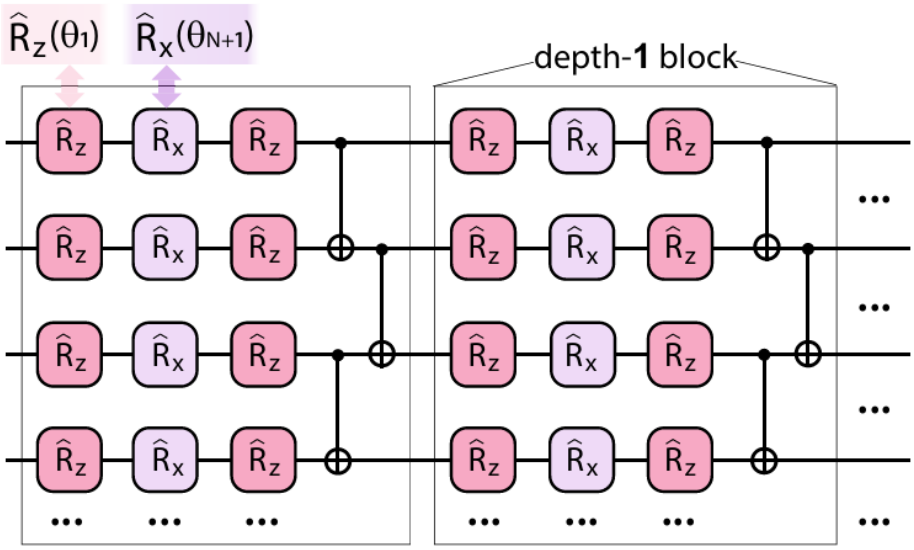

Figure 2.2: Hardware Efficient Ansatz (HEA). *The variational ansatz is implemented in a hardware-efficient form. It consists of a parameterized rotation layer following the $\hat{R}_z$–$\hat{R}_x$–$\hat{R}_z$ pattern, which allows for the realization of arbitrary single-qubit rotations. Each gate in the rotation layer is assigned its own variational angle. This layer is followed by an entangling layer composed of CNOT gates acting between nearest-neighbor qubits. The combination of rotation and entangling layers forms a single building block, and this block is repeated d times to construct the full variational circuit $\hat{U}_\theta$. Figure reproduced from [18].*

inherently defined by the featue map and input x. In this study, we stick to the definition of A. Manzano where circuit expressivity refers to its ability to approximate any function within a particular function class and domain up to arbitrary precision, with respect to a chosen metric. As DQCs with product-type feature maps introduce entanglement via the ansatz, the circuit flexibility can only be increased with depth (or qubits). Note that assessing the effect of entanglement on this flexibility is nontrivial. This is why we consider the role of entanglement in a more general sense. The ansatz we will use is the well-known **Hardware Efficient Anzatz** (HEA) depicted in Fig. 2.2. The HEA consists of layers of rotational gates interleaved with global entangling layers that assume a linear connectivity of the qubits on the hardware. This ansatz display is particularly NISQ-friendly as it uses native gates and connectives, minimizing the effect of noise. The rotations are arranged in a $\hat{R}_z$-$\hat{R}_x$-$\hat{R}_z$ sequence parametrized by independent angles θ. The block of rotations followed by *CNOTs* can be repeated d times implicitly increasing the depth of the circuit. The total

number of variational parameters θ therefore scales as $\mathcal{O}(3 \cdot N \cdot d)$ which can complicate the search for large N or d. The entanglement also grows with both d and N since there are $N-1$ $CNOT$ gates per layer d. However, for a fixed-size circuit, the HEA has a known saturation point after which the amount of entanglement does not increase anymore despite adding more layers: $d \approx N$ [23]. In this study, we will only consider DQCs with depth $d \leq N$ as increasing depth beyond saturation point will not significantly increase the amount of entanglement [3]. Note that increasing the DQC depth will introduce additional parameters that could improve the solution quality. However, since our main focus here is on the amount of entanglement required to accurately capture the solution of a given differential equation, we will not rely on the additional expressivity introduced by these extra parameters.
For the loss function, we use the MSE to match the differential equation $F[\cdot]$ with a net contribution of 0:

$$\mathcal{L}_{\theta}^{(\mathrm{diff})}[d_x f, f, x] = \frac{1}{M} \sum_{i=1}^{M} \left(F\left[d_x f(x_i), f(x_i), x_i\right] - 0\right)^2 \tag{2.5}$$

where we denote the differential loss over M training points as L_f. Note the boundary conditions are inherently taken into account within functions approximations when using the *floating boundary handling* strategy, as explained in Section 2.1.1.

2.1.3 Trainability & Generalization

An expressive PQC such as our DQCs can produce a wide range of trial solutions. The likelihood of finding the optimal solution heavily depends on the loss landscape (i.e. how the loss changes with parameters) but also on the optimizer exploring the space. Hence, the choice of the loss function (i.e. the target distance with which we want to approximate the true function), the optimizer type and its learning rate, should be carefully made in accordance with the quantum model. A quantum circuit with a higher expressivity, in the sense of M. Schuld et al., does not necessarily result in a higher chance of finding the optimal solution. Moreover, in specific circuit architectures and initialization regimes, increasing depth can cause gradients to vanish—a problem known as *barren plateaus* [32][20]. In fact, gradients exponentially decrease with depth which prevents optimizers to converge towards global minima [46][1]. Therefore, a balance between expressivity and trainability must be found. Note that gradient-free methods are also affected by the curse of barren plateaus [2]. We will use the

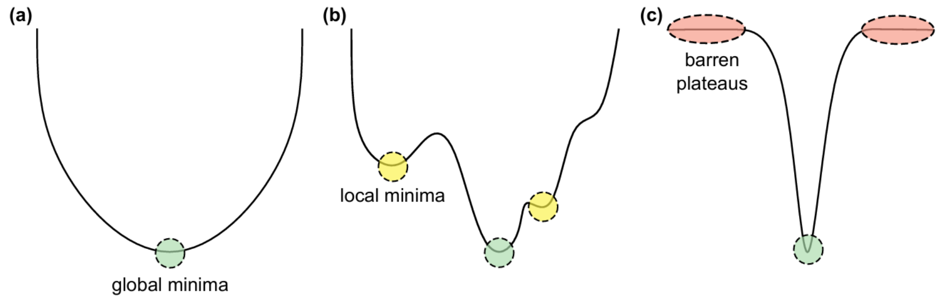

Figure 2.3: Typical Loss Landscapes. *a) Convex loss landscape where gradients at every point lead directly to the global minimum, making optimization straightforward. b) Rugged loss landscape where gradients may point away from the global minimum; local minima are common, where the loss is minimal only within a small region. c) Barren plateaus where gradients vanish exponentially, causing the optimization process to stall due to the lack of meaningful updates. Figure reproduced from [28].*

same optimizer as in the original work: the Adaptive Momentum Estimation (ADAM) algorithm which is standard for training neural networks. Like in A. Paine et al.'s work, we have not encountered the problem of barren plateaus at high depths. Illustrative loss landscapes can be found in Fig.2.3.
In the context of approximating solution functions to differential equations, a close-enough local minima might be sufficient to capture the essential features of the target functions. This is what we aim to achieve with an entanglement-constrained approximator. Finally, the generalization of PQCs is known to be affected by the loss function and the quantum model itself. An overly expressive model (in the number of parameters) will overfit the training points, failing to learn the input-output relationship. The loss function on the other hand might determine which information is being prioritized during training, leading to a biased model. This could very much depend on the training data as well. We will therefore consider two training grids for the reproduction of A. Paine et al.'s results: the Uniform grid (i.e. equidistant points) and the Chebyshev grid (i.e. Chebyshev nodes).

2.2 Matrix Product States (MPS)

Matrix Product States (MPS) offer a compact and structured representation of quantum states using sequences of low-rank tensors, making them particularly effective for systems with limited entanglement [42]. While traditional quantum circuits define state evolution through explicit gate sequences, MPS represent the state itself and implement gate operations through efficient local tensor updates. This allows them to simulate circuit behavior directly, without constructing the full Hilbert-space representation. The bond dimension determines the maximum entanglement entropy the MPS can encode, offering fine control over its expressiveness. When this dimension is sufficiently large, MPS can replicate the outputs of full quantum circuits exactly. However, due to differences in parameterization and function space, the resulting optimization trajectories may still diverge from those of equivalent circuit-based models.

2.2.1 Concept

MPS is a structured classical representation of quantum states, particularly efficient for systems with linear connectivity and limited entanglement. Rather than simulating quantum dynamics directly, MPS allows one to represent quantum states compactly and simulate the action of quantum gates through local tensor updates. An N-qubit pure state $|\psi\rangle$ can be expressed as:

$$|\psi\rangle = \sum_{s_1,\ldots,s_n=0}^{1} \sum_{\alpha_1,\ldots,\alpha_{n-1}=1}^{\chi} \mathbf{M}^{[1],s_1}_{1\alpha_1} \mathbf{M}^{[2],s_2}_{\alpha_1\alpha_2} \cdots \mathbf{M}^{[n-1],s_{n-1}}_{\alpha_{n-2}\alpha_{n-1}} \mathbf{M}^{[n],s_n}_{\alpha_{n-1}1} |s_1 s_2 \ldots s_n\rangle \tag{2.6}$$

where each tensor $\mathbf{M}^{[i],s_i}_{\alpha_i\alpha_{i+1}}$ encodes the local state of qubit i and its correlations with neighboring sites. These tensors are typically of size $\chi \times \chi$, where the bond dimension χ controls the amount of entanglement that can be represented between adjacent systems. Because of their local structure, MPS enables efficient simulation of quantum operations. A single-qubit gate acts on just one site, modifying the corresponding tensor $\mathbf{M}^{[i]}$ directly. Since each such tensor has χ^2 elements, the computational cost of updating it is $\mathcal{O}(\chi^2)$. A two-qubit gate acting on neighboring qubits i and $i+1$ requires updating two tensors, $\mathbf{M}^{[i]}$ and $\mathbf{M}^{[i+1]}$. These are first combined into a single tensor by contracting over the shared index α_i, yielding a larger object of size $(\chi \cdot 2) \times (\chi \cdot 2)$. The gate is applied to the data encoded in these dimensions, and the resulting tensor is then reshaped and decomposed into two updated tensors, restoring the MPS structure. This procedure requires $\mathcal{O}(\chi^3)$ operations and typically increases the size of the shared bond dimension from χ to at most 2χ, reflecting the increase in entanglement introduced by the gate [43]. The growth of χ determines how

much information needs to be passed between the two parts of the system, and it governs the complexity of subsequent operations. Thus, the bond dimension plays a central role in controlling both the expressivity and the computational cost of MPS-based simulations.
The MPS can also be interpeted as a chain of $N-1$ successive Schimdt decompositions [27], over all possible bipartitions $[1\cdots i]$:$[(i+1)\cdots N]$, each yielding a set of Schmidt coefficients λ_α and orthonormal basis states $|\xi_\alpha\rangle_A$, $|\eta_\alpha\rangle_B$, such that:

$$|\psi\rangle = \sum_{\alpha=1}^{\chi_A} \lambda_\alpha |\xi_\alpha\rangle_A \otimes |\eta_\alpha\rangle_B \tag{2.7}$$

The Schmidt rank λ_α reflects the entanglement between partitions A and B, and the maximum over all such bipartitions defines the bond dimension $\chi = \max\limits_{A} \chi_A$, which governs the expressive power of the MPS representation.

2.2.2 Entanglement Entropy

The Von Neumann *entanglement entropy* is a popular measure for the amount of entanglement a quantum state exhibits. For a quantum state $p = |\psi\rangle\langle\psi|$, the entanglement entropy of the subsystem A with reduced density matrix $\rho_A = Tr_B[\rho]$ is:

$$S(\rho_A) = -\mathrm{Tr}[\rho_A \log \rho_A] \tag{2.8}$$

Combining Eq. 2.7 and Eq. 2.8, we obtain:

$$S(\rho_A) = -\sum_{\alpha=1}^{\chi} \lambda_\alpha^2 \log \lambda_\alpha^2 \tag{2.9}$$

Hence, whenever A and B are in a product state, $S(\rho_A)$ becomes 0, while when they are maximally entangled $S(\rho_A)$ becomes $n\_A * log(2)$ (n_A being the number of qubits in subsystem A here). Note that the shared entanglement entropy between the two subsystems is always equal: $S(\rho_A) = S(\rho_B)$.

2.2.3 Bond dimension & Truncation

The bond dimension χ refers to the maximal Schmidt rank across all bipartitions and governs the amount of entanglement that the MPS can faithfully represent. In practice, a sufficiently large χ is required to exactly encode the state $|\psi\rangle$, but many physically relevant quantum states have rapidly decaying Schmidt spectra—meaning that only a small number of the largest Schmidt coefficients λ_α carry significant weight in determining the state's entanglement structure. This enables the use of truncated Schmidt decompositions, where only the leading $\tilde{\chi} < \chi$ coefficients are retained, reducing computational cost while preserving a high-fidelity approximation of the quantum state. In this context, the truncated entanglement entropy captures the entanglement retained after discarding small Schmidt coefficients. If $\tilde{\chi} < \chi$ denotes the number of dominant terms retained, the entropy becomes:

$$S(\rho_A) = -\sum_{\alpha=1}^{\tilde{\chi}} \lambda_\alpha^2 \log \lambda_\alpha^2 \tag{2.10}$$

This provides a measure of the entanglement preserved under truncation and is commonly used to quantify approximation quality in tensor network simulations.
MPS-based simulation is most efficient when the quantum state remains slightly entangled, meaning that the required bond dimension χ grows at most polynomially with system size N. In such cases, both memory usage and computational time scale as $\text{poly}(N)$, enabling efficient classical simulation of the quantum evolution.
A key factor governing this efficiency is how entanglement grows within the system. In one-dimensional gapped systems, entanglement entropy typically obeys an *area law*, meaning that it saturates and remains approximately constant regardless of subsystem size [16]. In contrast, *volume-law entanglement* arises when entropy grows proportionally to the number of qubits in the subsystem, reaching a maximum at the central bipartition around $N/2$. In the context of our DQCs, the Hardware-Efficient Ansatz (HEA) generates volume-law entanglement due to its chains of entangling $CNOT$ gates, which propagate correlations across the system after approximately $N/2$ layers [24]. To ensure faithful MPS-based simulation, we restrict the bond dimension to $\chi \leq 2^{N/2}$, which suffices to capture the maximum entanglement present in the circuit without artificially enhancing model capacity. This constraint ensures that comparisons focus solely on the amount of entanglement required to represent accurate solutions to differential equations, avoiding confounding effects from overparameterized tensor representations.

2.3 Related Studies

This study's first objective is to recover results from A. Paine et al. with DQCs. To our knowledge, there are no published work that explicitly tries to recover these latter. Nonetheless, as DQCs fall under the category of PQCs, many addressed their expressivity and learnability in general. A major contribution to the field is the work by E. Grant et al. on initialization strategy for addressing *barren plateaus* in PQCs [13]. They demonstrate that random initialization in PQCs often lead to barren plateaus—regions where gradients vanish exponentially with the circuit size (number of qubits N)—making training ineffective. This result directly implies that DQCs, which rely on gradient-based training, exhibit some dependence on initialization where a poor initial parametrization can prevent the circuit from learning anything meaningful. Taking this into account, this study addresses DQCs expressivity and learnability with respect to the examples from the original DQC paper over different initial parametrizations.
Then, the second objective is to recover DQCs results using MPS with maximal bond dimension. A relevant work is the one from G. Vidal showing that any quantum circuits can be exactly expressed, up to numerical precision, as a sequential application of isometric[1] tensors, leading to an MPS form with bond dimension $\chi = 2^{N/2}$. However, MPS being distinct models with respective sets of parameters, there is no guarantee they will converge towards the same solution when trained identically to DQCs. This study addresses MPS performance against DQCs in solving differential equations.
Finally, the third objective is to recover DQCs results with MPS truncation. Specifically, we seek for the minimum amount of entanglement required to accurately capture solutions to differential equations by virtue of reducing the bond dimension. K. Chen et al. adopt a similar approach on distributed quantum neural networks (QNNs) for distributed photonic quantum computing where QNNs are used to generate high-dimensional probability distributions, which are then mapped to classical neural network weights via a trained MPS model with certain bond dimension [8]. They empirically demonstrate that an MPS truncation from $\chi = 10$ down to $\chi = 4$ achieves not less than 3% the classification accuracy at maximal

[1] An isometric tensor preserves norm when projecting a lower-dimensional space into a higher-dimensional one (i.e. maintain normalization locally when being reshaped and contracted), which is essential for efficient and stable simulation of quantum states.

bond dimension $\chi = 10$ for the MNIST[2] dataset. Moreover, they conduct ablation studies confirming the importance of minimum effective entanglement in representational capacity of the QNN-to-CNN mapping. This study investigates the minimum bond dimension required to accurately capture solutions to differential equations with respect to DQCs. Ultimately, the goal is to empirically analyze and explain the functional role of entanglement in circuit expressivity in the sense of A. Manzano et al. [26]. We aim to determine which characteristic features necessitate entanglement in order to construct differential systems that would benefit most from quantum.

[2]MNIST is a benchmark dataset widely used in machine learning, especially for image classification tasks.

Chapter 3

Methods

The experimental design is guided by the ultimate objective of determining how entanglement influences the quality of function approximation, which functional features remain inaccessible without it, and which classes of DEs give rise to features that, in turn, necessitate entanglement for accurate approximation. To investigate these questions, we consider three differential equations from the original work on DQCs by A. Paine et al. [18]: a damped harmonic oscillator, a system of strongly coupled differential equations, and a Navier–Stokes instance. These cases are selected to span a progression from linear to nonlinear systems, from smooth to stiff dynamics, and from non-correlated to correlated behavior, by systematically increasing the relevant control parameters. The final example, involving Navier–Stokes dynamics, opens the route towards exploring multidimensional and potentially turbulent regimes. Note that optimal control or stochastic problems are excluded from the scope of this study, and have been extensively addressed in other works by the DQC research team (see [29][19]).

3.1 Approximating Solution Functions with DQCs

Approach The first step is to reproduce the results reported in A. Paine et al.'s work under identical settings. For each of the three differential equation examples, the same DQCs are used: Chebyshev and Chebyshev tower feature maps, a hardware-efficient ansatz (HEA) with a linear entangling architecture, identical circuit depth, number of qubits, and cost operator—specifically, the total magnetization operator $\hat{C} = \sum_{j=1}^{N} \hat{Z}_j$. The training procedure also mirrors the one from the original paper, employing the `ADAM` optimizer with the same learning rate (when provided), the same loss function, and the same number of epochs. All ingredients are further described in Section 2.1.2. While the DQC research team uses an equidistant training grid for all cases, Chebyshev nodes are also considered here, due to their expected compatibility with Chebyshev-based feature maps [28]. Performance is evaluated using two metrics: the *quality loss* $L_q = \frac{1}{M}\sum_{i=1}^{M}[f(x_i) - u(x_i)]^2$ and the *differential loss* $L_f = \mathcal{L}_\theta[d_x f, f, x]$, in order to assess whether comparable loss values to those reported are achieved. While we generally consider solutions with $L_f < 10^{-1}$ to be acceptable, the original paper often reports values closer to 10^{-2}, i.e. $L_f \ll 10^{-1}$, which can have a significant impact on the predicted solution quality L_q. Although it is numerically true that $10^{-1} - 10^{-100} \approx 10^{-1}$, these values correspond to qualitatively different regimes: a solution with $L_f \approx 10^{-1}$ often yields noticeable deviations in the predicted function $f(x)$ compared to the true solution $u(x)$, whereas lower differential loss values tend to correlate with much more accurate generalization. To evaluate the expressivity and learnability of these DQCs under fixed configuration, simulations are repeated for 10 random initializations of the variational parameters θ_{init}. This helps assess the sensitivity of convergence and solution quality to initial parametrization. Since the original work reports consistent performance across increasing control parameters—which introduce nonlinearity and stiffness—we adopt the same progressive control parameter scaling. In the event that the reference results are not reproducible under any of the tested initializations, a reasonably exhaustive hyperparameter search is performed. However, since the central focus of this study is the role of entanglement in accurately solving differential equations, classical hyperparameters (e.g., optimizer, epochs) are kept fixed wherever possible. Instead, we explore increasing the circuit depth (and number of qubits when needed) to scale entanglement entropy. This strategy avoids relying on the classical optimizer to overcome expressivity limitations and allows a more direct analysis of how entanglement contributes to solution quality.

Setup Simulations are performed using `Qadence`, a quantum circuit library developed by PASQAL [37], which integrates with PyTorch's autograd and supports differentiable circuit components. Gradients are computed using automatic differentiation (autodiff), while the *parameter-shift rule* (PSR) is also implemented on a noiseless statevector simulator for theoretical comparison. In autodiff, gradients are calculated by reverse-mode differentiation through the simulator's computational graph, requiring exactly one backward pass per training step, regardless of the number of parameters. For a batch of M input samples, M forward passes are performed to compute the loss on each input; these losses are then aggregated (e.g., summed), and a single backward pass is used to compute gradients of the total loss with respect to all parameters. The computational graph is dynamically constructed during each forward pass, but its structure remains identical across inputs when the circuit architecture is fixed. In contrast, PSR evaluates the gradient for each parameter of the encoding gates using the general analytic formula $\frac{\partial}{\partial\theta}\langle O\rangle = \frac{1}{2}\left[\langle O\rangle_{\theta+\frac{\pi}{2}} - \langle O\rangle_{\theta-\frac{\pi}{2}}\right]$ which requires two independent forward circuit evaluations per parameter, per input (see also Section 2.1.2). Thus, for a batch of M samples, PSR requires $2N \cdot M$ forward passes per training step. Both methods produce analytically equivalent gradients under noiseless simulation with compatible gate generators as shown in M. Schuld et al.'s work [35]. Autodiff is used throughout this study due to its significantly lower computational cost and native integration with the simulation backend.

Experiments

1. **Damped Oscillator**: is a single first-order DE (ODE) that is linear in the unknown function u, and u nonlinear in the variable x. Setting the boundary condition sets the solution to the problem ($u(0) = 1$):

$$\frac{du}{dx} + \lambda u\left(\kappa + \tan(\lambda x)\right) = 0, \quad u(0) = u_0 \tag{3.1}$$

   where the solution $u(x)$ is a damped oscillating function with k controlling the decay, and λ the oscillations:

$$u(x) = \exp(-\kappa\lambda x)\cos(\lambda x) \tag{3.2}$$

   First, we compare the solution quality when using Uniform and Chebyshev grids over 10 initial parametrizations. Just like in the original work, we set $k = 0.1$ and $\lambda = 8$. For the second experiment, we solve the ODE for the harder case where $\lambda = 20$ with the best-found input grid. Finally, for k fixed, we gradually increase the oscillatory factor $\lambda = [2\pi, 8, 12, 16, 20]$ in order to meticulously assess the DQC generalization to nonlinear, stiff systems.

2. **Strongly Coupled Oscillators**: is a system of two ODEs (with boundary conditions $u_1(0) = 0.5$ and $u_2(0) = 0$, respectively) coupled by

$\lambda_{1,2}$ as the following:

$$\begin{aligned} F_1[d_x u, \mathbf{u}, x] &= \frac{du_1}{dx} - \lambda_1 u_2 - \lambda_2 u_1 = 0, \quad u_1(0) = u_{1,0} \\ F_2[d_x u, \mathbf{u}, x] &= \frac{du_2}{dx} + \lambda_2 u_2 + \lambda_1 u_1 = 0, \quad u_2(0) = u_{2,0} \end{aligned} \tag{3.3}$$

where the larger $|\lambda_1|$ is compared to $|\lambda_2|$ the more strongly coupled the two equations are. We use two separate DQCs of same structure with independent parameter sets. Hence, we double the number of parameters that must be optimized simultaneously. As the analytical solution is not provided in the original paper, we derive ourselves the analytical form in the Appendix A.2. We obtain the following solutions:

$$\begin{aligned} u_1(x) &= 0.375 \sin(4x) + 0.5 \cos(4x) \\ u_2(x) &= -0.625 \sin(4x) \end{aligned} \tag{3.4}$$

First, we try to recover the same results under 10 different initializations for $\lambda_1 = 5$ and $\lambda_2 = 3$. Then, with the same DQCs possibly, we gradually increase the coupling strength $\lambda_1 = [5, 6, 9, 12, 15]$ in order to assess the DQC generalization to even-stronger correlated functions.

3. **Navier-Stokes (stationary)**: is a system of three highly nonlinear ODEs. This instance of the Navier-Stokes equations is the quasi-1D approximation of an inviscid flow passing through a convergent-divergent nozzle (see Fig 3.1). Considering a fluid motion subject to Newton's law, we can write the problem in terms of the continuity, energy conservation, and momentum conservation equations:

$$\begin{aligned} \frac{\partial \rho}{\partial t} &= -\rho \frac{\partial V}{\partial x} - \rho V \frac{\partial (\ln A)}{\partial x} - V \frac{\partial \rho}{\partial x} \\ \frac{\partial T}{\partial t} &= -V \frac{\partial T}{\partial x} - (\gamma - 1) T \left(\frac{\partial V}{\partial x} + V \frac{\partial (\ln A)}{\partial x} \right) \\ \frac{\partial V}{\partial t} &= -V \frac{\partial V}{\partial x} - \frac{1}{\gamma} \left(\frac{\partial T}{\partial x} + \frac{T}{\rho} \frac{\partial \rho}{\partial x} \right) \end{aligned} \tag{3.5}$$

where ρ stands for density, T for temperature, and V for velocity. The heat capacities ratio γ equals 1.4, and the spatial nozzle shape $A(x) = 1 + 4.95(2x-1)^2, \quad 0 \le x \le 1$. We solve the stationary-state version of the problem by setting the initial conditions to: $\rho(x = 0) = 1$, $T(x = 0) = 1$, and $V(x = 0) = 0.1$. The non-dynamical version of the problem remains of interest as the steady-state solution cannot

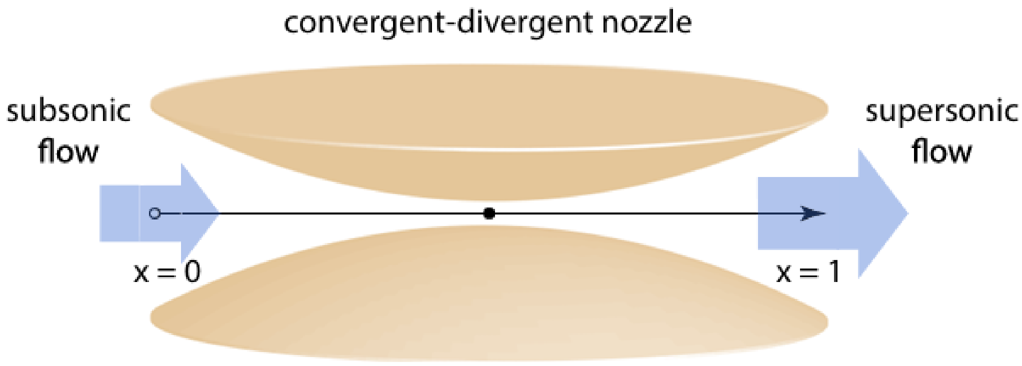

Figure 3.1: Quasi-1D Fluid Dynamics. *We consider a fluid dynamics problem involving a convergent-divergent nozzle. Air flows through the converging section of the nozzle (for $x < 0.5$), passes through the throat located at the center, and exits through the diverging section (for $x > 0.5$). The system models quasi-1D flow of an inviscid fluid, where we track the evolution of density, temperature, and velocity via the continuity, energy conservation, and momentum conservation equations collectively known as Navier-Stokes. Figure reproduced from [18].*

be easily retrieved by classical solvers due to a significant singularity at $V(x) = \sqrt{T(x)}$. The system becomes also very stiff near the nozzle. For these reasons, we have not been able to compute a reference solution. Just like in the original work, we use the gradient information as a quality measure here.

The problem is solved in two training stages to avoid the divergent point at $x = 0.5$. First, 20 points from $(0, 0.4)$ are trained for 200 epochs. Then, 40 points among which 20 belongs to the interval $(0, 0.4)$ and the remaining 20 to $(0.6, 0.9)$ are passed along the optimal parameters from the first training stage for 600 epochs. With those points, 20 more points from the first domain $(0, 0.4)$ are passed with 5 additional points from the range $(0.6, 0.9)$ as weak regularization to interpolate the solution around the nozzle. The regularization points contribute to the loss function as $\mathcal{L}_\theta^{(\text{reg})}[f, x] = \sum_{r=1}^{R} \zeta(n_j) \left(f_\theta(x_{\text{reg},r}) - u_{\text{reg},r}\right)^2$ and slowly vanishes around epoch 150. Hence, $\zeta(n_j) = 1 - \tanh\left(\frac{n_j - n_{\text{drop}}}{\delta_j n_{\text{iter}}}\right)$ where n_{drop} is set to 150 and n_{iter} to 600. Note the original results are produced with *Chebyshev feature map* and not *Chebyshev tower feature map* like in the other examples. We therefore compare solution quality between the two for the same set of initial parametrizations.

3.2 Assessing Functional Equivalence between DQCs and MPS

Approach & Experiments In theory, MPS with maximal bond dimension should be able to simulate DQCs exactly for a given input and set of parameters, up to floating-point precision. Given a quantum state $|\psi_\theta(x)\rangle$, the final measurement under the same observable O should lead to the same approximation: $\langle MPS_D(x)|O|MPS_D(x)\rangle = f(x) = \langle\psi_\theta(x)|O|\psi_\theta(x)\rangle$ where bond dimension $D = 2^{N/2}$ captures all the necessary entanglement entropy from the original circuit. However, the MPS representation comes with its own set of classical parameters (and function space). In that sense, MPS does not necessarily converge towards the same solution when being trained similarly to the DQC it represents. We will therefore assess how their respective convergences differ under identical settings. In other words, can MPS still retrieve a good solution for a DQC state that lead to a good solution? We demonstrate it for the original **Damped Oscillator** equation where $\lambda = 8$ over 10 initial parametrizations.

Setup Simulations are performed using `TensorCircuit`, a quantum machine learning library developed by Zhang et al. [47], which integrates with JAX's XLA-accelerated backend for automatic differentiation and supports differentiable MPS representations. This framework enables efficient and fast gradient computation for single MPS-based evaluations. However, for each input sample M, a new MPS must be instantiated based on the newly computed parameters, resulting in a significant computational and memory overhead when scaled over multiple epochs. To mitigate this, JAX's support for GPU-parallelized function and gradient evaluation is leveraged. All simulations are executed on the Academic Leiden Interdisciplinary Cluster Environment (ALICE) at Leiden University [22], which provides NVIDIA A100 40 GB GPUs.

3.3 Approximating Solution Functions with MPS Truncation

Approach & Setup The central objective of this study is to explore entanglement constrained approximations of solution functions to differential equations. Specifically, we ask: How much entanglement is needed to capture the essential features of a solution function accurately, as measured by the final training loss (L_f) and the generalization error (L_q) across

the domain? Using MPS, we seek for the minium bond dimension that still lead good enough solutions, as quantified by the performance metrics defined above. Starting with bond dimension $D = 2^{N/2}$, we systematically decrease the bond dimension down to $D = 1$, corresponding to a product state with zero entanglement entropy ($log_2(1) = 0$ - see Section 2.2.2). Under the assumption that certain features of complex differential equations inherently rely on nontrivial entanglement, we seek to identify which properties degrade as the entangled structure decays. Ultimately, this line of investigation aims to inform the design of differential equations that stand to benefit the most from quantum.
Simulations are performed with `TensorCircuit` on ALICE HPC cluster's GPU nodes. A table containing the used memory and compute resources is given in the Appendix A.4.

Experiments We consider the same 3 examples from the previous experiments: the **Damped Oscillator**, the **Strongly-Coupled Oscillators**, and the stationary **Navier-Stokes** equations. Considering the same set of control parameters, we reduce bond dimension for each and retain the best-found solution over 10 different parametrizations of the corresponding DQC.

Chapter 4

Results

4.1 RQ1: How effective are DQCs in terms of expressivity and learnability?

4.1.1 Damped Oscillator

For the first experiment, we reproduce A. Paine et al.'s results with identical DQCs: 6 qubits encoded with Chebyshev tower feature map, HEA of depth 5, and total magnetization cost operator. We train them with `ADAM` for 250 epochs on both Uniform and Chebyshev grids of 20 points each where $x \in (0, 0.9)$. The original paper claims to find solutions with $L_f < 10^{-1}$ and $L_q < 10^{-4}$. For $\lambda = 8$, only $1/50$ initial parametrizations lead to a solution with L_f and L_q of the same order for the Uniform grid. Whereas for the Chebyshev grid, we retrieve $6/50$ matching solutions. Note that our performance metric might be overly strict. In practice, one could be completely satisfied with a solution quality of $L_q < 10^{-2}$, depending on the use-case. In this particular case, we get $24/50$ and $45/50$ good parametrizations for Uniform grid and Chebyshev grid, respectively. We show the best-found solution for Chebyshev grid in Fig. 4.1. Using the Chebyshev grid, we tried to solve the equation for $\lambda = 20$. We show our best-found approximation over 50 initial parametrizations in Fig. 4.2. We were unable to recover a single good solution under the same settings as in $\lambda = 8$. We tried several DQCs up to 12 qubits, depth 11, and different learning rates with additional epochs. We also applied classical rescaling $f_\theta(x) = \alpha \langle x | f_\theta \rangle$, which was introduced in a related work [30]. None of these tentatives leads to a better solution than the one in Fig.4.2 (at least

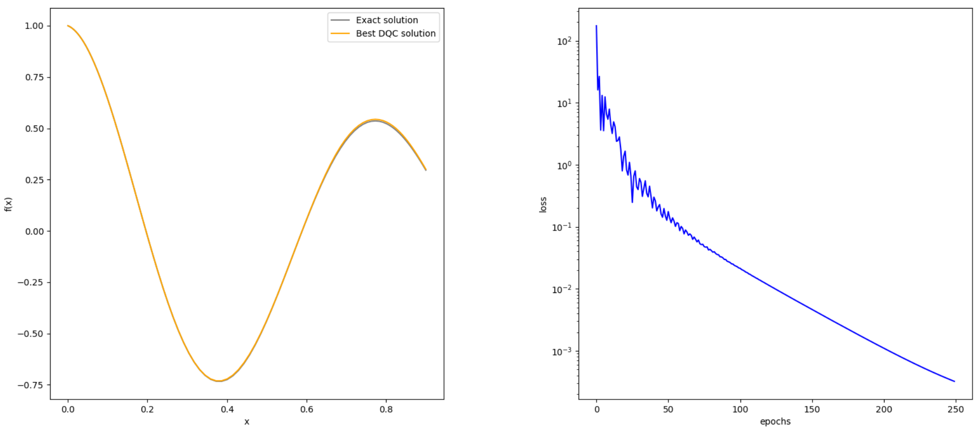

Figure 4.1: *Best-found solution for the* ***Damped Oscillator*** *with Chebyshev Grid over* 50 *initial parametrizations ($\lambda = 8$). The prediction is evaluated on* 100 *testing Chebyshev nodes from* $(0, 0.9)$. *We use* `ADAM` *with $lr = 0.1$ The solution has $L_f = 0.000322$ and $L_q = 0.000019$ and perfectly matches the reference solution.*

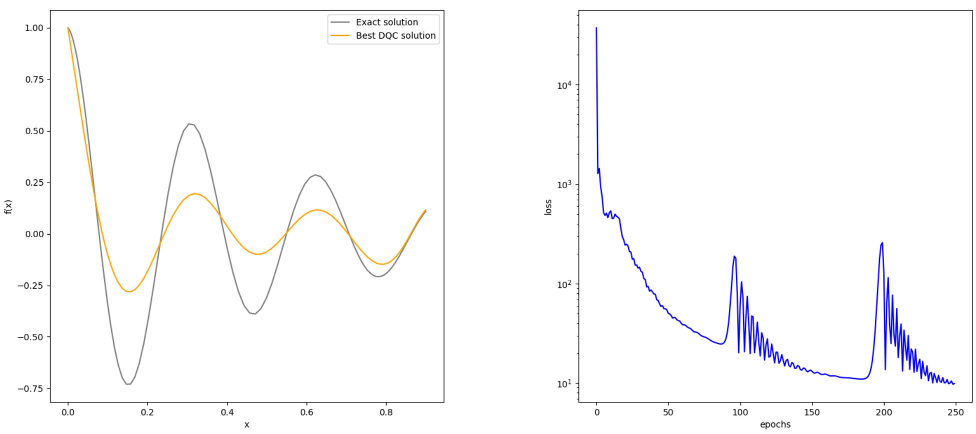

Figure 4.2: *Best-found solution for the* ***Damped Oscillator*** *with Chebyshev Grid over* 50 *initial parametrizations ($\lambda = 20$). The solution has $L_f = 9.881482$, $L_q = 0.03$, $lr = 0.031389$. The model fails in approximating the target function accurately in terms of amplitude, but still catches the nonlinear shape.*

for the same 50 initializations). It is in fact not that surprising when taking a closer look at the ODE: $\frac{du}{dx} + \lambda u(\kappa + \tan(\lambda x)) = 0$ where $\tan(\lambda x)$ introduces increasingly many singularities at $x = \frac{\pi}{2\lambda} + \frac{n\pi}{\lambda}$, $n \in \mathbb{Z}$, for higher λ within the training range. This results in a function with extremely steep gradients around these singularities and accentuates the stiffness arising from the interaction between fast oscillating and slowly decaying modes in the solution —- namely, $cos(\lambda x)$ and $exp(-k\lambda x)$. We use as a stiffness indicator the following index derived by C.W. Gear [12]:

$$S(x) = \left| \frac{u''(x)}{u'(x)} \right| \tag{4.1}$$

where large values correspond to regions where very small changes in input cause large variations in output. We derive the index for Eq. 3.1 in the Appendix A.1, and plot it over 100 points in $(0, 0.9)$ (see Fig. 4.3).

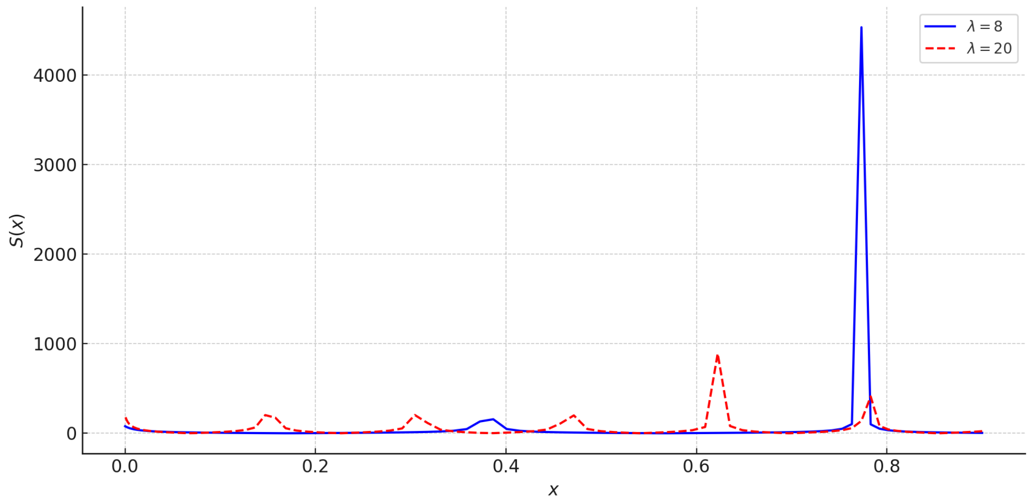

Figure 4.3: *Stiff-Index profiles for the* ***Damped Oscillator*** *at $\lambda = 8$ and $\lambda = 20$. For $\lambda = 20$, the system exhibits stiff locations at $x = \{0.18, 0.32, 0.45, 0.63\}$ approximately. For $\lambda = 8$, there are only two locations at $x = \{0.39, 0.79\}$ approximately, where stiffness is significantly greater at $x = 0.79$ than any other peaks observed for both cases. The model struggles in predicting the solution around around that point.*

We obtain 2 peaks for $\lambda = 8$ at $x = \{0.39, 0.79\}$ approximately, and 5 smaller peaks for $\lambda = 20$ at $x = \{0.18, 0.32, 0.45, 0.63\}$ approximately.
For the purpose of this study, we consider another version of the **Damped Oscillator** from a related work by A. Paine et al. [30] that does not depend

on such singular terms (where k is set to 1 with initial condition $u(0) = 1$):

$$\frac{du}{dx} + \kappa u + \lambda \exp(-\kappa x) \sin(\lambda x) = 0 \tag{4.2}$$

and with solution:

$$u(x) = \exp(-\kappa x) \cos(\lambda x) \tag{4.3}$$

We obtain 10/10 and 5/10 good solutions at both $\lambda = 8$ and $\lambda = 20$, respectively. For $\lambda = 8$, we use 5 qubits, depth 4, and 100 epochs. For $\lambda = 20$, we use 7 qubits, depth 6, and 100 epochs as well. The rest of both settings is identical to the previous **Damped Oscillator** example. We plot the best-found solutions in Fig. 4.4. Note the slight mismatch at ≈ 0.8 for $\lambda = 8$, which can be attributed to the pronounced stiffness-induced peak.

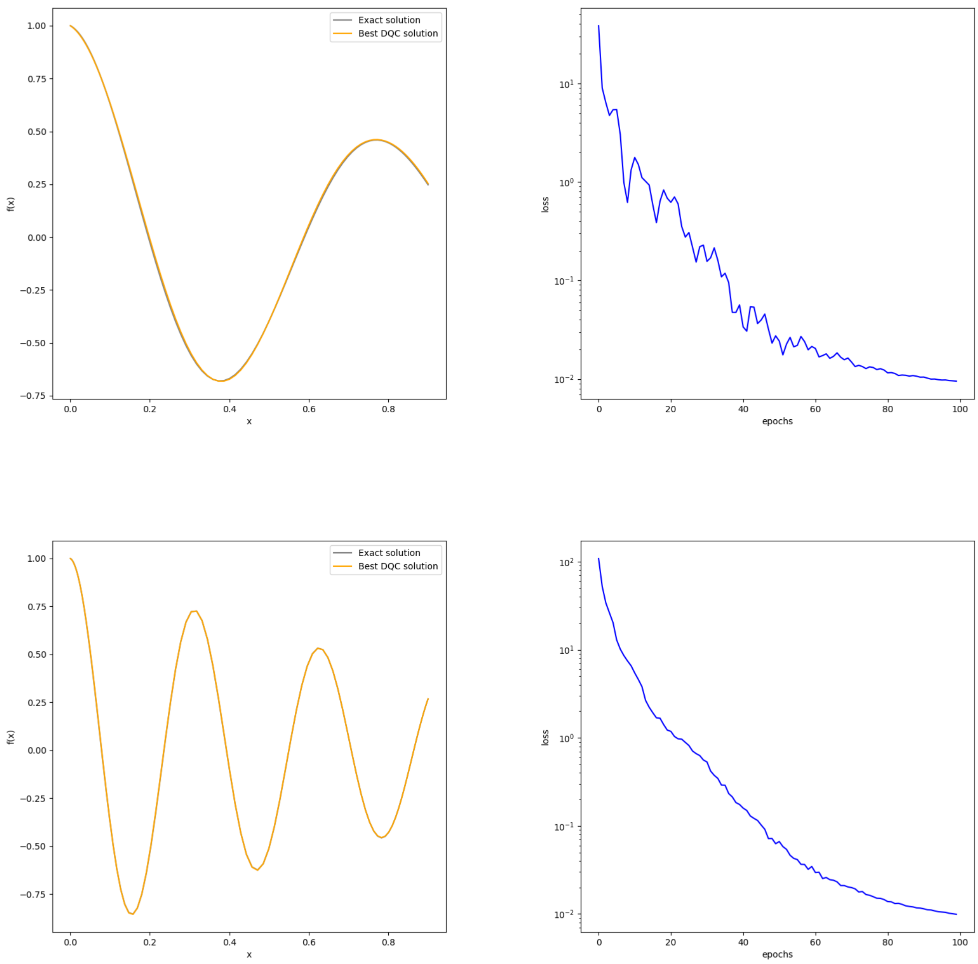

Figure 4.4: *Best-found solutions for the new* ***Damped Oscillator*** *with* $\lambda = 8$ *and* $\lambda = 20$ *over* 10 *initial parametrizations. We use* `ADAM` *with* $lr = 0.1$. *For* $\lambda = 8$, *the solution has* $L_f = 0.000925$ *and* $L_q = 0.000002$. *For* $\lambda = 20$, *the solution has* $L_f = 0.011706$ *and* $L_q = 0.000017$. *The model accurately represent the target functions for both cases.*

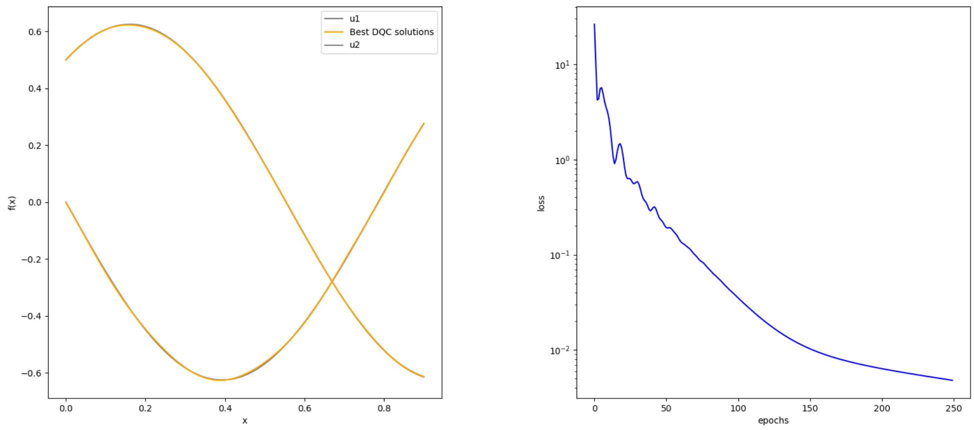

Figure 4.5: *Best-found solution for the* ***Strongly Coupled Oscillators*** *over* 10 *initial parametrizations ($\lambda_1 = 5$, $\lambda_2 = 3$). The prediction is evaluated on* 100 *testing Chebyshev nodes from* $(0, 0.9)$. *We use* `ADAM` *with $lr = 0.02$. The solution has $L_f = 0.004806$, $L_q^1 = 0.000002$, and $L_q^2 = 0.000005$, which perfectly matches the reference solution.*

4.1.2 Strongly Coupled Oscillators

We reproduce A. Paine et al.'s results for the two strongly coupled ODEs with identical DQCs: 6 qubits encoded with Chebyshev tower feature map, HEA of depth 5, and total magnetization cost operator. We train them with `ADAM` for 250 epochs on Chebyshev grid of 20 points where $x \in (0, 0.9)$. For this problem, the learning rate is set to 0.02, and $\lambda_{1,2}$ to 5 and 3, respectively. The original paper claims to find solutions with $L_f < 10^{-2}$ and $L_q < 10^{-4}$. For 10 initial parametrizations, we retrieve 7 solutions with L_f and L_q of the same order. We show the best-found solution in Fig. 4.5. We have not been able to solve the coupled system for higher coupling strengths $\lambda_1 = [6, 9, 12, 15]$ after trying multiple configurations of qubits, depth, and learning rate with more epochs. In the best case, DQCs accurately captures the solution for the first differential equation but constantly misfit the other. We show the best-found solution for $\lambda_1 = 6$ in Fig. 4.6. Note that for each λ_1, we have to recompute the analytical solution from scratch.

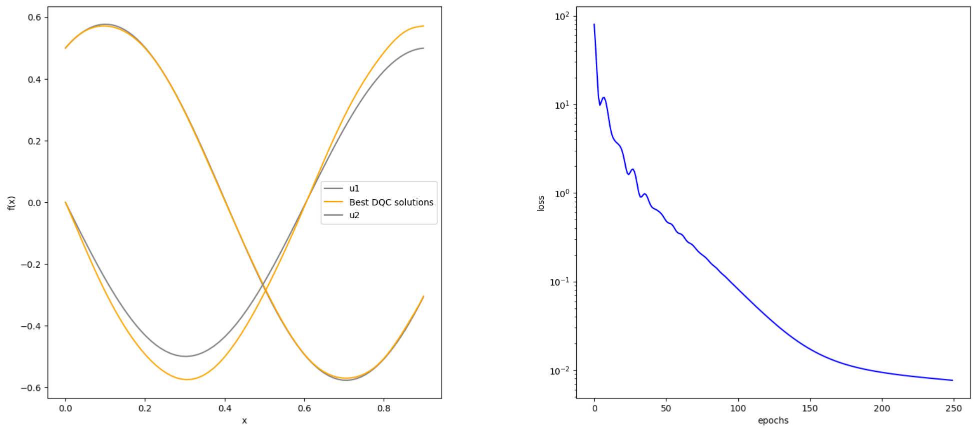

Figure 4.6: *Best-found solution for the* ***Strongly Coupled Oscillators*** *over* 10 *initial parametrizations ($\lambda_1 = 6$, $\lambda_2 = 3$). We use* `ADAM` *with $lr = 0.02$. The solution has $L_f = 0.007676$, $L_q^1 = 0.000011$, and $L_q^2 = 0.002704$. The model constantly fail in approximating the second target function u_2 for all initial parametrizations.*

4.1.3 Navier-Stokes (stationary)

We reproduce A. Paine et al's results for the fluid dynamics problem with identical DQCs and training stages: 6 qubits encoded with Chebyshev feature map, HEA of depth 6, and total magnetization cost operator. We train the models with `ADAM` in two stages. For the first stage, we avoid the divergent point of the nozzle at $x = 0.5$ and consider 20 training points from the interval $(0, 0.4)$. The learning rate is set to 0.01, and the number of epochs to 200. For the second training stage, we feed the variational angles from stage 1 as initial parameters for stage 2 training. We consider 20 training points from the interval $(0, 0.4)$ and 20 more points from $(0.6, 0.9)$. For weak regularization, we use 20 points in $(0, 0.4)$ benefiting from the previously found solution combined with 5 additional points from $(0.6, 0.9)$, representing weak bias towards supersonic solution type. The learning rate is set to 0.005, and the number of epochs to 600 with regularization up to epoch 150. The original paper claims to find solutions with final $L_f < 10^{-1}$. For 10 initial parametrizations, we retrieve 5 solutions with $L_f \approx 10^{-1}$ (so not $< 10^{-1}$). We show the best-found solution after each training stages in Fig. 4.7. We also show the best-found solution when using the more expressive Chebyshev tower feature map in Fig. 4.8. Surprisingly, the Chebyshev tower feature map struggles in approximating function derivatives that match the desired nonlinear contributions. For

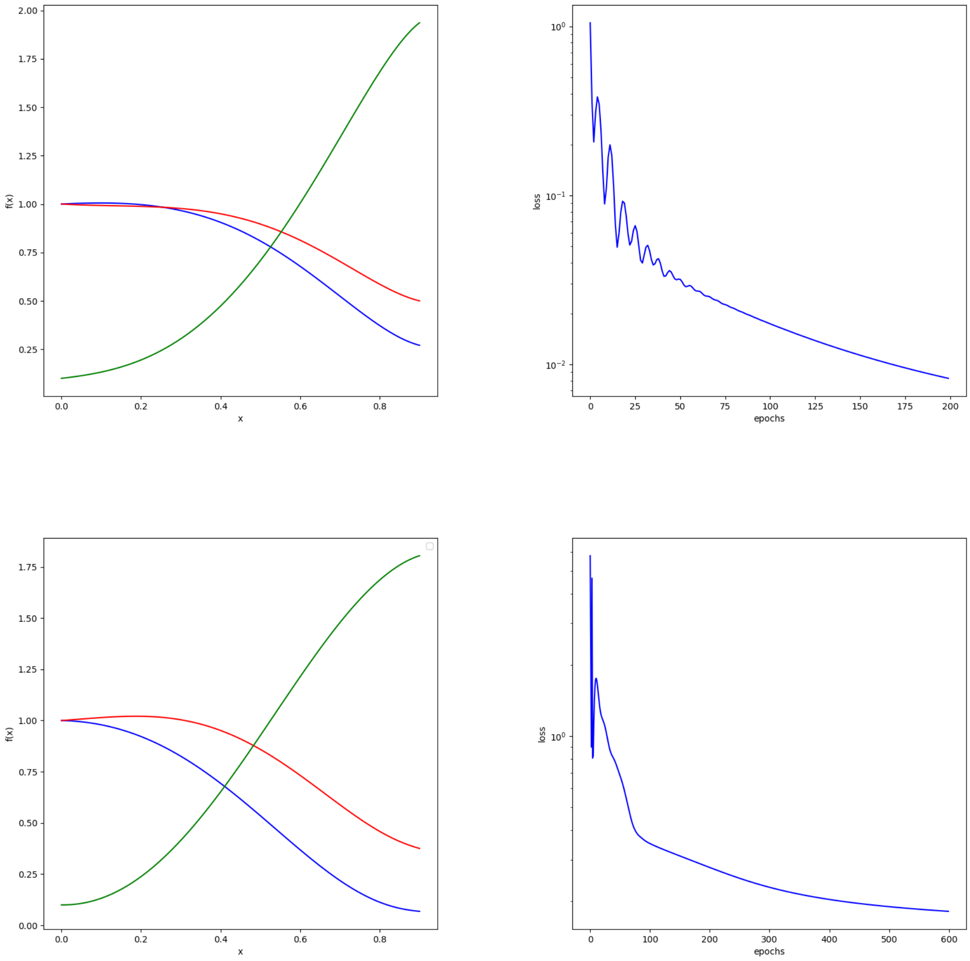

Figure 4.7: *Best-found solution for the stationary* ***Navier-Stokes*** *with the standard Chebyshev feature map over* 10 *initial parametrizations. For stage 1, the solution has* $L_f = 0.011339$*. After stage 2, the solution's differential loss becomes* $L_f = 0.185075$*. Note the second training loss is greater as it extrapolates the whole domain over the nozzle. The best-found solution is one order higher than the reference solution.*

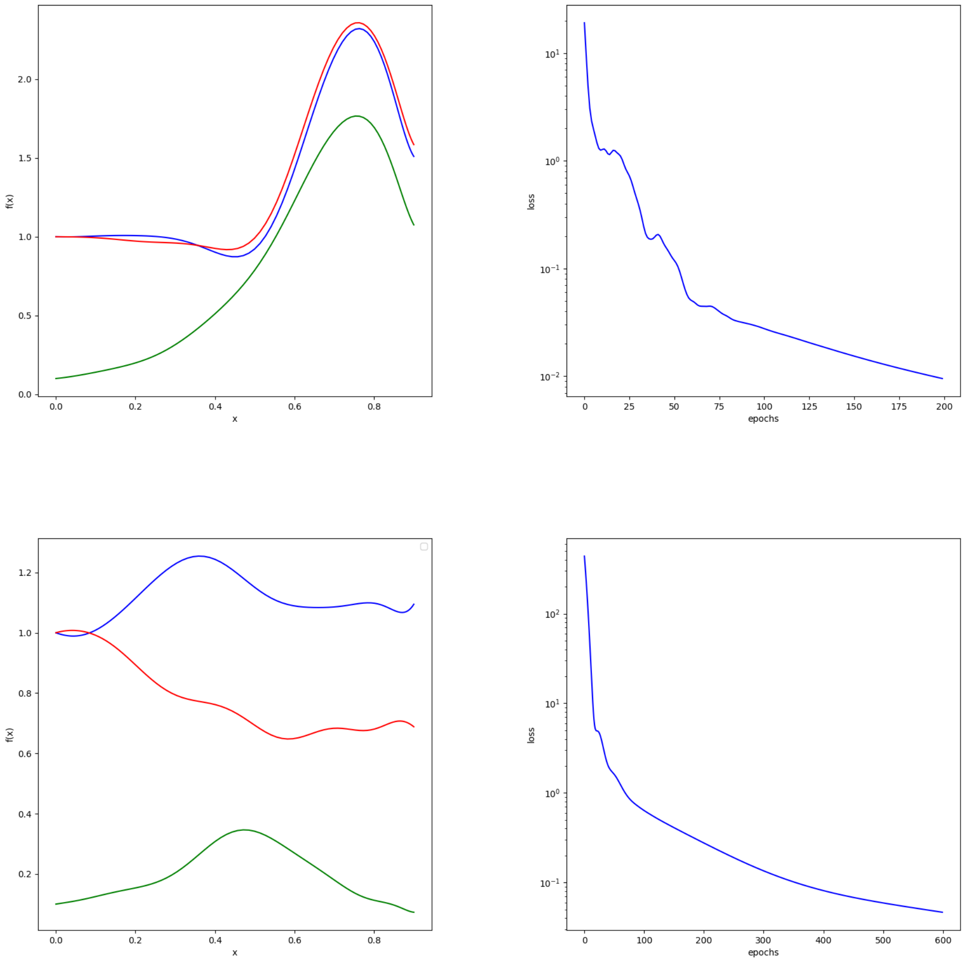

Figure 4.8: *Best-found solution for the stationary* ***Navier-Stokes*** *with the Chebyshev tower feature map over* 10 *initial parametrizations. For stage 1, the solution has* $L_f = 0.0153757$*. After stage 2, the solution's differential loss becomes* $L_f = 0.052251$*. The model fails completely in approximating the target functions, despite matching the differential equation.*

10 initial parametrizations, we have not been able to retrieve a single solution that represents the sought functions. We noticed a mismatch between `Qadence`'s inbuilt Chebyshev Tower and the corresponding description in the original paper. The DQC paper defines the Chebyshev tower feature map as $\hat{U}_{\varphi}(x) = \bigotimes_{j=1}^{N} \hat{R}_{y,j}\,(2j \cdot \arccos(x))$ where a single-qubit rotational gate introduces Chebyshev polynomials of degree j. However, `Qadence` seems to define the nonlinear mapping as $\varphi(x) = j \cdot arcos(x)$, which we demonstrate in our codebase 6. We discuss the potential consequences in Section 5.1.

4.2 RQ2: To what extent can MPS-based simulation recover DQC results with maximal bond dimension?

For the original **Damped Oscillator** from A. Paine et al.'s work, we demonstrate that MPS can lead to a good solution for a DQC state that lead to a good solution over 10 circuit parametrizations. We display the final predictions in Fig. 4.9, and show the computed differential and quality losses every 10 epochs for one initialization in the following table:

	L_f (DQC)	L_f (MPS)	L_q (DQC)	L_q (MPS)
0	739.595947	742.873047	0.405680	0.405358
10	4.038614	2.970488	0.041316	0.043542
20	2.240279	2.239892	0.062509	0.069657
30	0.660982	0.789965	0.031619	0.037274
40	0.360996	0.365831	0.014486	0.013721
50	0.134787	0.113711	0.006427	0.004358
60	0.081543	0.067466	0.005448	0.002581
70	0.029791	0.013712	0.002740	0.001538
80	0.010003	0.046699	0.001195	0.001480
90	0.012038	0.081324	0.001037	0.001283
100	0.011415	0.031404	0.000837	0.001135
110	0.019154	0.008215	0.001019	0.001611
120	0.022856	0.209747	0.000905	0.002312
130	0.001478	0.273612	0.000904	0.002276
140	0.001360	0.015833	0.000846	0.003630
150	0.021841	0.040383	0.000814	0.001573
160	0.039430	0.004331	0.000767	0.001647
170	0.052073	0.015953	0.000471	0.001442
180	0.028505	0.019831	0.000407	0.000676
190	0.029802	0.046161	0.000386	0.001079
200	0.023520	0.123212	0.000438	0.001273
210	0.002315	0.096397	0.000767	0.000991
220	0.008022	0.023559	0.000493	0.001778
230	0.001093	0.014758	0.000471	0.001505
240	0.001316	0.004883	0.000564	0.001971
Final	**0.004433**	**0.127921**	**0.000302**	**0.000746**

At epoch 0, the DQC has $L_f = 739.595947$ and $L_q = 0.405680$ whereas for the corresponding MPS with maximal bond dimension, $L_f = 742.873047$

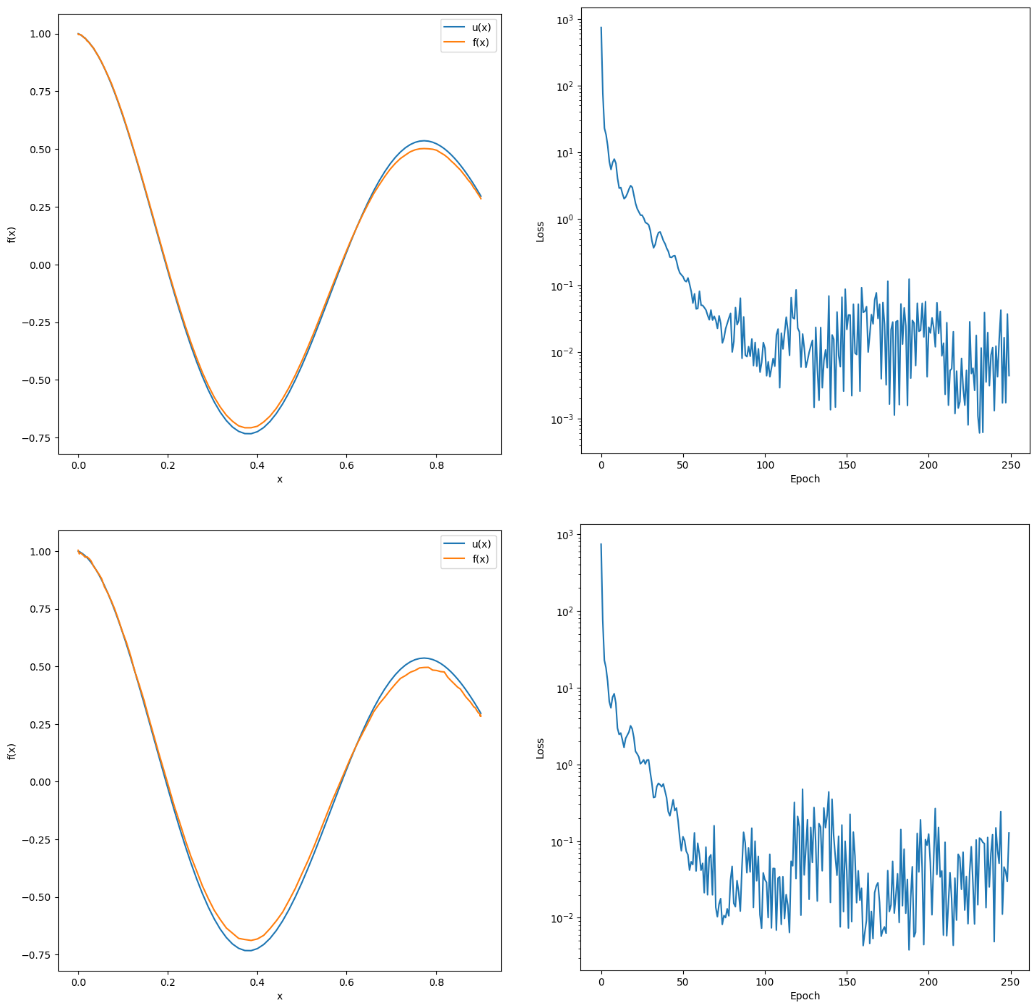

Figure 4.9: *Similar predictions for the original* ***Damped Oscillator*** *between DQC and MPS with maximal bond dimension. The approximations coincide at epoch* 0*, and diverge throughout the training process. Despite converging towards significantly different derivatives representations, the two models produce almost identical approximations up to floating-point precision. For DQC, the solution after* 250 *epochs has* $L_f = 0.004433$ *and* $L_q = 0.127921$*, whereas for MPS, the solution has* $L_f = 0.000302$ *and* $L_q = 0.000746$.

and $L_q = 0.405358$. Both differential and quality losses are of the same order, respectively. The quality losses appear to be closer (up to 10^{-3}) than the differential losses in this case. This shows that MPS with sufficient amount of entanglement can faithfully represent the DQC approximation up to floating-point precision. At epoch 10, the quality losses already start to deviate from each other through iterations up to $10^{-}2$. At epoch 240, they significantly differ at $L_q = 0.000564$ for DQC and $L_q = 0.001971$ for MPS. Similarly, differential losses match at epoch 0 and starts to diverge throughout the training. Interestingly, their final predictions, with $L_q = 0.000302$ for DQC and $L_q = 0.000746$ for MPS, match up to $10^{-}3$ as depicted in Fig. 4.9.

We also provide the predictions for another initialization which exhibits similar training behaviors in Fig. 4.10. However, the final predictions significantly differ this time with $L_q = 0.000636$ for DQC and $L_q = 0.011700$ for MPS.

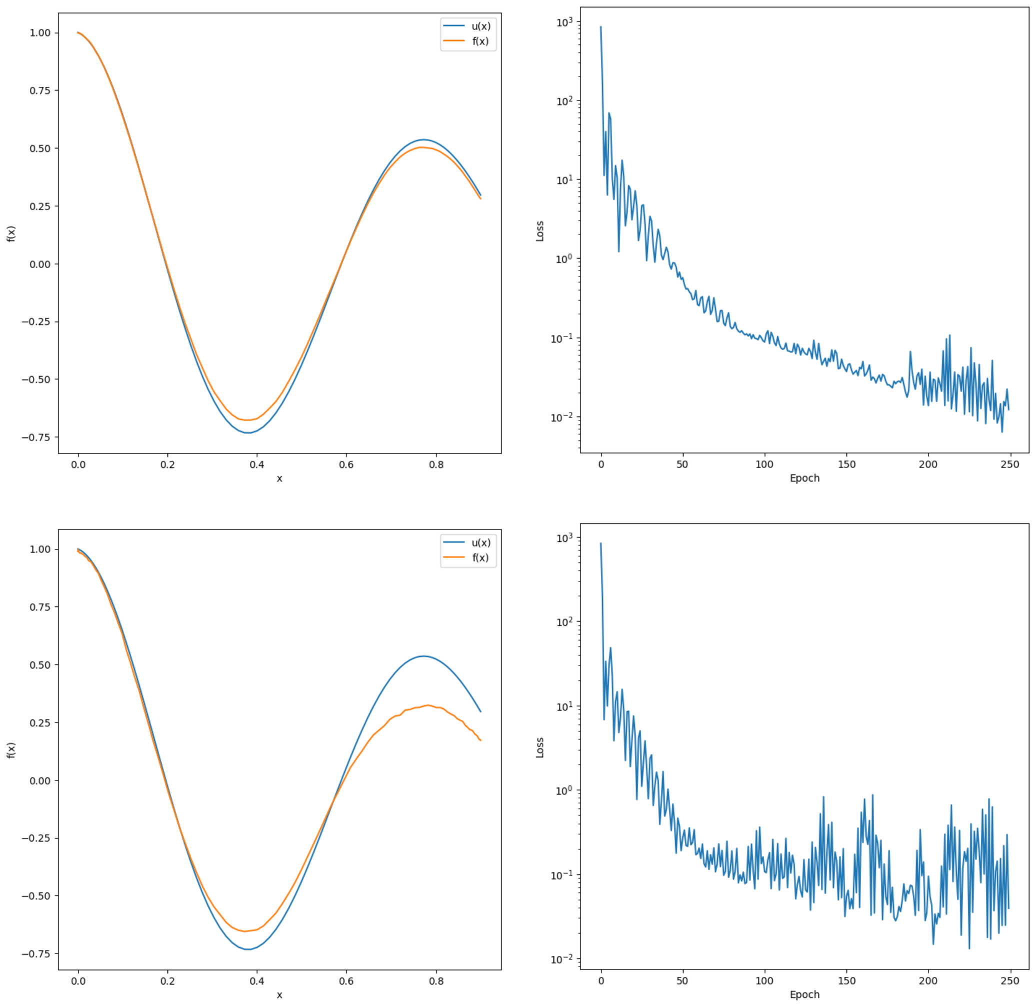

Figure 4.10: *Different predictions for the original* ***Damped Oscillator*** *between DQC and MPS with maximal bond dimension. The approximations coincide at epoch 0, and diverge throughout the training process. The two distinct representations converge towards significantly different outputs, where $L_f = 0.012184$ and $L_q = 0.000636$ for the DQC, whereas $L_f = 0.039197$ and $L_q = 0.011700$ for the corresponding MPS.*

4.3 RQ3: What is the minium bond dimension required to accurately capture the solution of a given differential equation?

4.3.1 Damped Oscillator

For the new **Damped Oscillator** equation introduced in Section 4.1.1, we decrease the bond dimension from $D = 2^{N/2}$ down to $D = 1$ for $\lambda = [2\pi, 8, 10, 12, 14]$ over 10 initial parametrizations. We provide the different DQCs specifications along the associated maximal bond dimensions in the following table:

	N qubits	depth d	$D_{\max} = 2^{\lfloor N/2 \rfloor}$
$\lambda = 2\pi$	4	3	4
$\lambda = 8$	5	4	4
$\lambda = 10$	5	4	4
$\lambda = 12$	6	5	8
$\lambda = 14$	6	5	8

Table 4.1: *DQCs architectures and the corresponding maximal bond dimensions. For N odd, the maximal bond dimension that captures all the entanglement from the DQC state is bounded by the size of the smallest subsystem. Hence, we 'floor' the fractional exponent* $\lfloor \frac{N}{2} \rfloor$.

For $\lambda = 2\pi$ with bond dimension $D = 4$, we obtain $6/10$ good solutions with respect to our performance metrics. By systematically decreasing the bond dimension, we find $4/10$ good solutions at $D = 2$ with respect to the same performance metrics. The best-found solutions for both $D = 4$ and $D = 2$ are given in Fig.4.11. For $D = 4$, we have $L_f = 0.007100$ and $L_q = 0.000021$ whereas for $D = 2$, we have $L'_f = 0.003215$ and $L'_q = 0.000007$. Note that $L'_f < L_f$ and $L'_q < L_q$. In Fig. 4.12, we display the worst-found result over the 10 parametrizations for $D = 2$ along the best one for $D = 1$. For $D = 2$, the worst-found solution has $L_f = 0.481617$ and $L_q = 0.096872$ whereas for $D = 1$, the best-found solution is one order higher for both $L_f = 2.298783$ and $L_q = 0.213234$.
For $\lambda = 8$ with bond dimension $D = 4$, we also obtain $6/10$ good solutions with respect to our performance metrics. By systematically decreasing the bond dimension, we find only $1/10$ good solution at $D = 2$ with respect to the same performance metrics. For $D = 4$, we have $L_f = 0.003533$

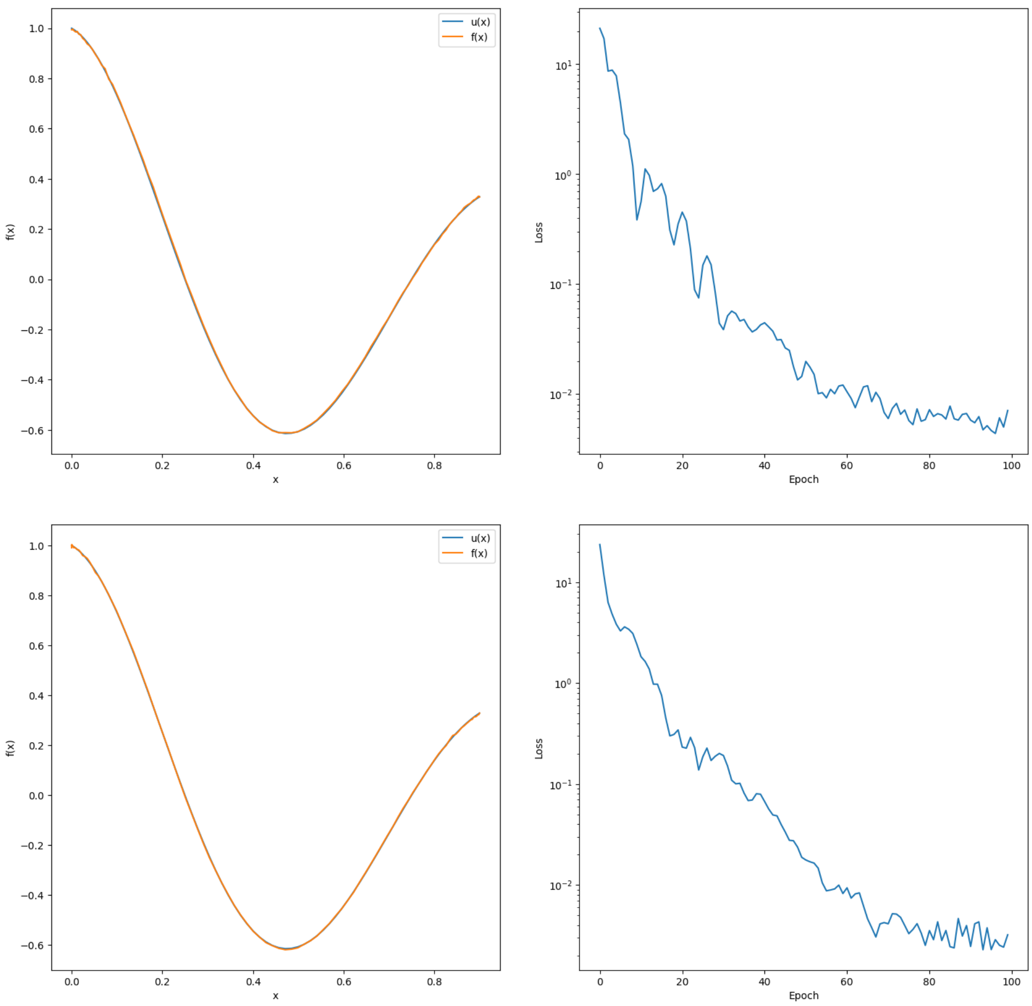

Figure 4.11: *Best-found solutions for the new **Damped Oscillator** with $\lambda = 2\pi$ and bond dimension truncation $4 \to 2$. Both model are trained identically over 10 initial parametrizations. At maximal bond dimension $D = 4$, the solution has $L_f = 0.007100$ and $L_q = 0.000021$, whereas at $D = 2$, the best-found solution has $L'_f = 0.003215$ and $L'_q = 0.000007$. The truncated model still lead to similarly good approximations.*

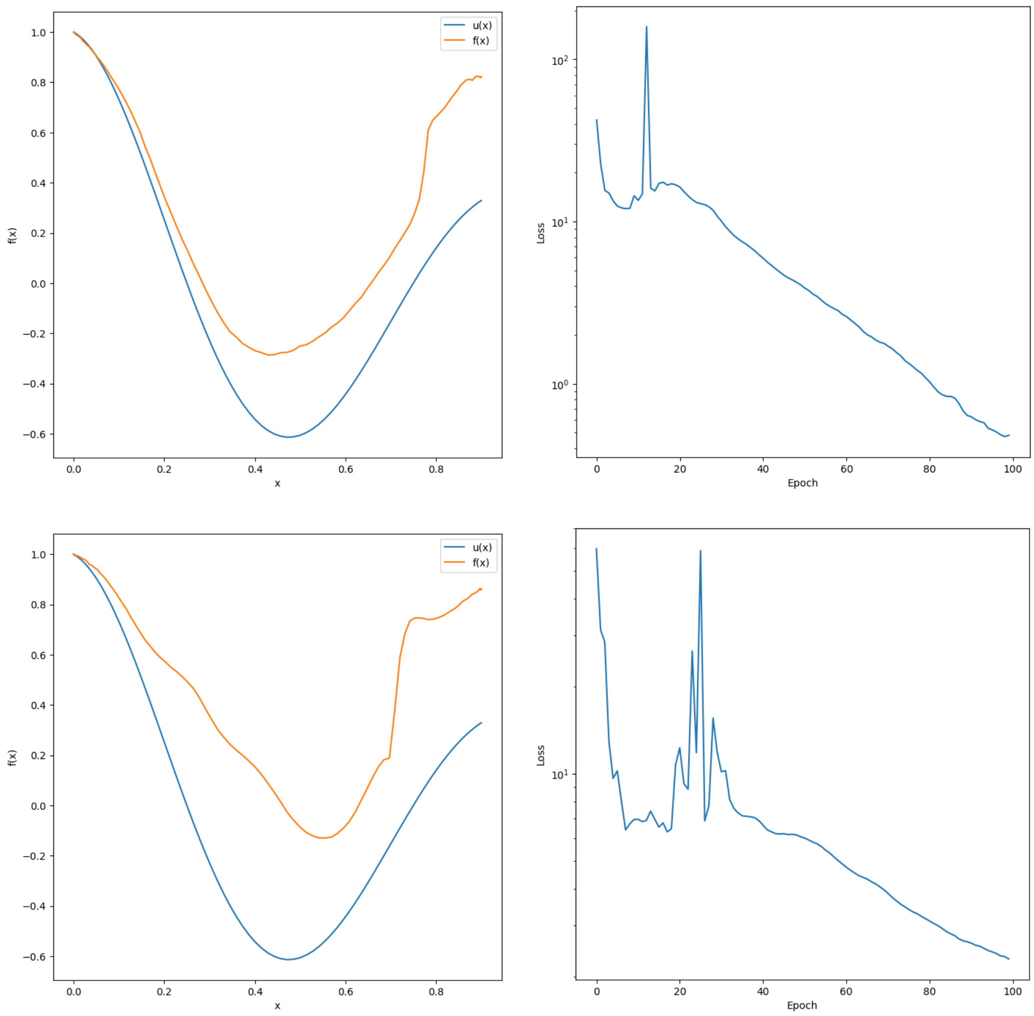

Figure 4.12: *Worst-found solution for the new* ***Damped Oscillator*** *with $\lambda = 2\pi$ at $D = 2$ against best-found solution at $D = 1$. At $D = 2$, the solution has $L_f = 0.481617$ and $L_q = 0.096872$, whereas at $D = 1$, the best-found solution has $L_f = 2.298783$ and $L_q = 0.213234$. The solution at $D = 1$ still captures the shape of the nonlinear curve of the target function.*

and $L_q = 0.000014$ whereas for $D = 2$, we have $L'_f = 0.013472$ and $L'_q = 0.000041$. Note that $L'_f \not< L_f$ and $L'_q \not< L_q$ this time. On the other hand for $\lambda = 10$, we have not found a truncation that lead to a good solution with respect to our performance metrics. At maximal bond dimension $D = 4$, we obtain 6/10 good solutions with best-found metrics $L_f = 0.017433$ and $L_q = 0.000060$. However, the closest solution among the bond dimensions and parametrizations we consider is at $D = 2$ (and not at $D = 3$) where $L_f = 0.017385$ and $L_q = 0.000114$. Note that the corresponding L_q is relatively near the 10^{-4} boundary.
For $\lambda = 12$ with bond dimension $D = 8$, we obtain 3/10 good solutions with respect to our performance metrics where the best-found solution has $L_f = 0.009268$ and $L_q = 0.000055$. By systematically decreasing the bond dimension, we find a good solution at $D = 6$ with $L'_f = 0.016486$ and $L'_q = 0.000090$. For $\lambda = 14$, we obtain only 2/10 good solutions with best-found $L_f = 0.017816$ and $L_q = 0.000063$ at $D = 8$, and $L'_f = 0.005216$ and $L'_q = 0.000089$ at $D = 4$ this time. Note that for $\lambda = 12$, we always find good solutions when decreasing the bond dimension until $D = 6$ (for our 10 parametrizations at least) whereas for $\lambda = 14$, there are no good solutions at $D = 5$ but one appears at $D = 4$. We display the best-found result for $D = 4$ against the best-found result for $D = 5$ in Fig. 4.13. We also display the best-found solution for $D = 1$ against the previously found solution for $\lambda = 2\pi$ at $D = 1$ in Fig. 4.14.

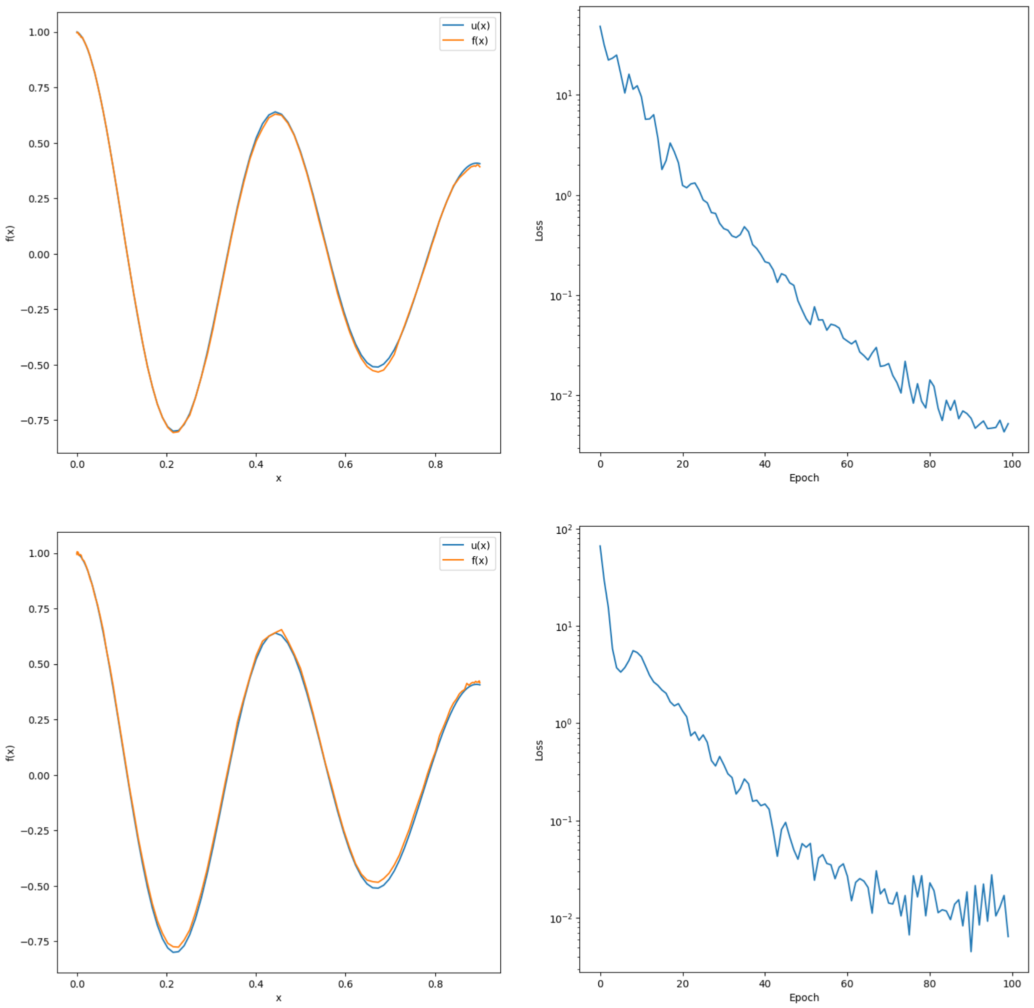

Figure 4.13: *Best-found solutions for the new* ***Damped Oscillator*** *with $\lambda = 14$ at $D = 4$ and $D = 5$. At $D = 4$, the solution has $L_f = 0.005216$ and $L_q = 0.000089$, whereas at $D = 5$, the best-found solution has $L_f = 0.006435$ and $L_q = 0.000271$. The solution at $D = 5$ does not meet our performance standards.*

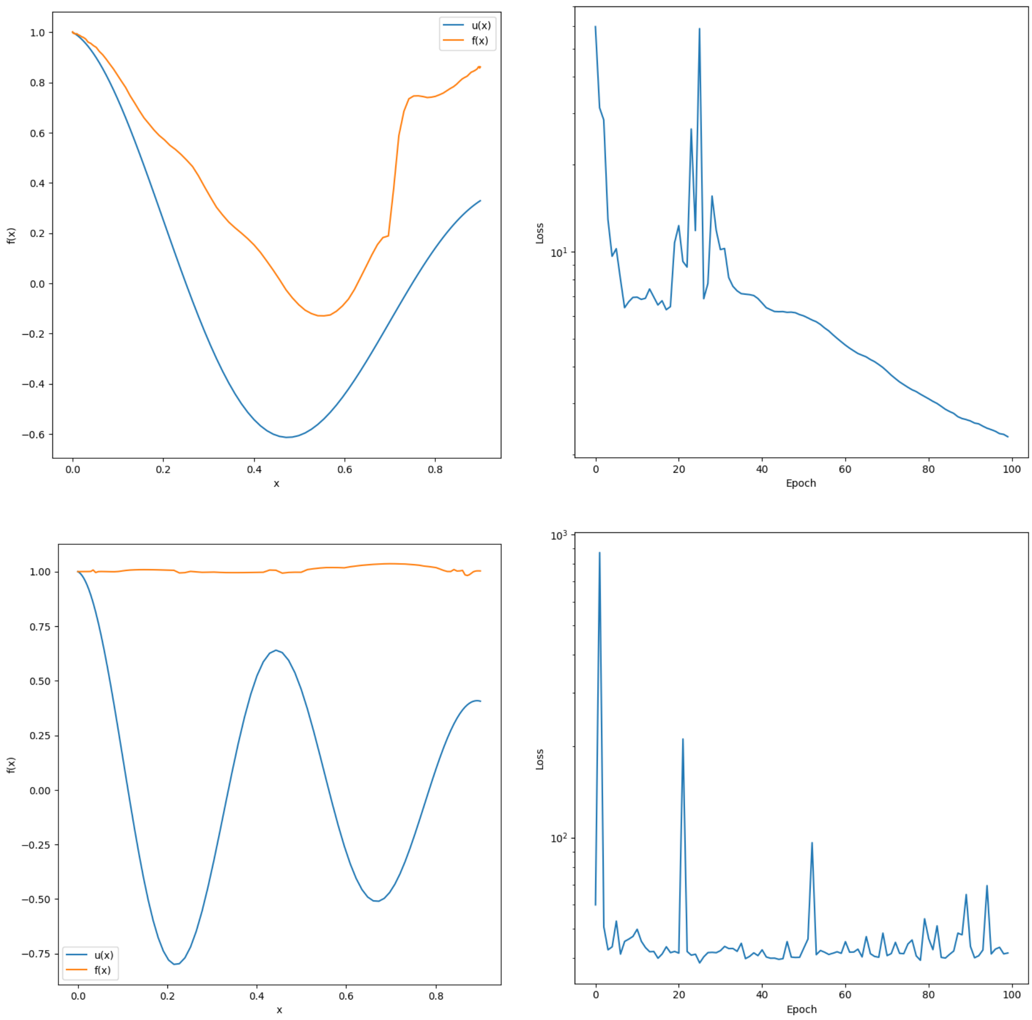

Figure 4.14: *Best-found solutions for the new* ***Damped Oscillator*** *with $\lambda = 14$ and $\lambda = 2\pi$ at $D = 1$. For $\lambda = 2\pi$, the solution still captures the shape of the nonlinear curve of the target function, whereas as for $\lambda = 14$, the model completely fails in converging towards a meaningful approximation as depicted by the horizontal line at initial boundary $u(0) = 1$.*

As we managed to recover A. Paine et al.'s results for the original **Damped Oscillator** at $\lambda = 8$ in Section 4.1.1, we perform MPS truncation from $D = 8$ down to $D = 1$. At $D = 8$, we do not obtain a single good solution with respect to our performance metrics. The system being much more stiff, we have also not been able to find a single good solution for any of the truncations and parametrizations we consider. We show the best-found result for $D = 8$ against the overall second best in Fig. 4.15.

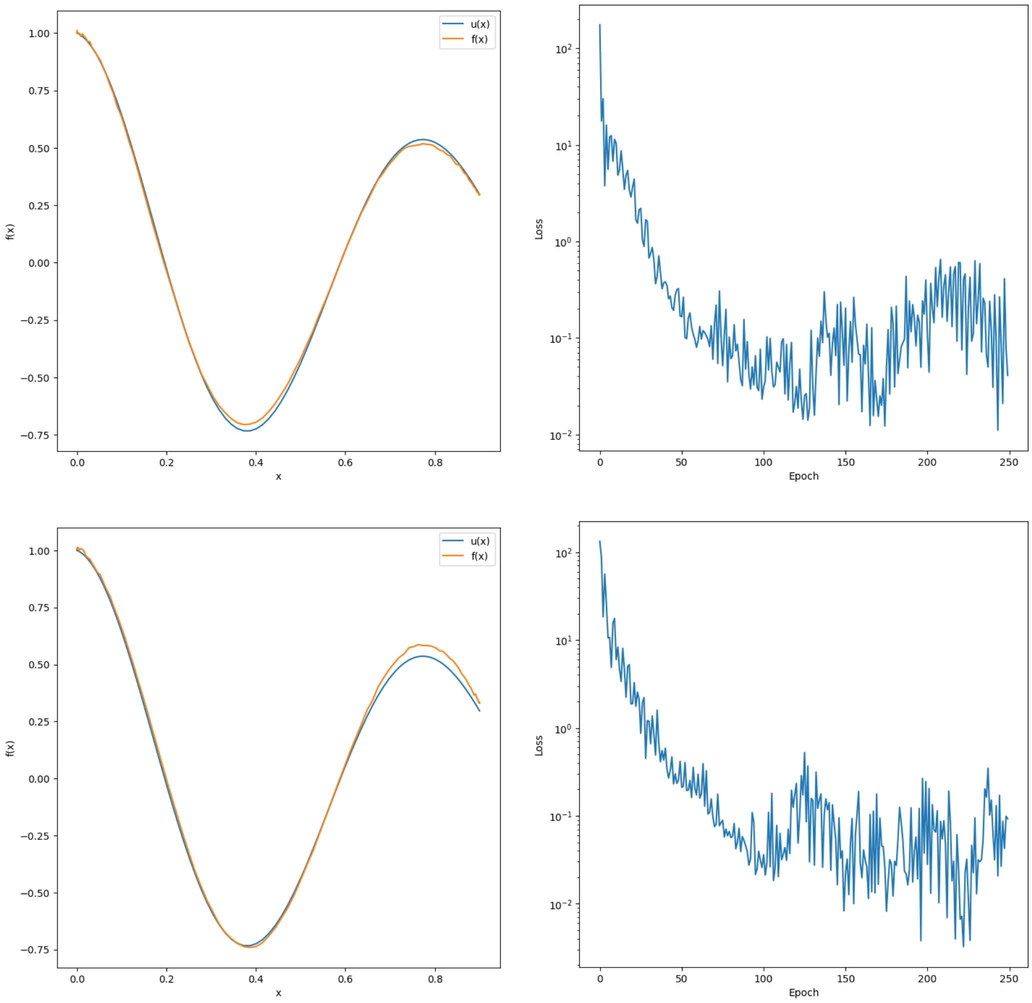

Figure 4.15: *Best-found solution for the original* **Damped Oscillator** *with* $\lambda = 8$ *at* $D = 8$ *with* $L_f = 0.040949$ *and* $L_q = 0.000199$. *The overall best-found truncation is at* $D = 7$ *where* $L_f = 0.092587$ *and* $L_q = 0.000791$. *Note that both solution and truncation do not quite meet our performance standards.*

4.3.2 Strongly Coupled Oscillators

As we did not manage to solve the equations for higher coupling strengths, we perfom MPS truncation for $\lambda_1 = 5$ only, where the maximal bond dimension is set to 8 (6 qubits). We decrease the bond dimension from $D = 8$ down to $D = 1$, and seek for good-enough approximations with respect to our performance metrics: $L_f < 10^{-2}$ and $L_q < 10^{-4}$. For $D = 8$, we obtain $2/10$ solutions with L_f and L_q of the same order. However, two other parametrizations lead to relatively good approximations with $L_q < 10^{-4}$, but with L_f slightly above 10^{-2} ($\sim$ 0.01 to be precise). We show one of these two solutions against the best-found result in Fig. 4.16. By systematically decreasing the bond dimension, we have not found a single good truncation with respect to our performance metrics (at least for our 10 initial parametrizations). The solution quality gradually decreases as the entanglement structure decays. We show the overall best-found truncation which is at $D = 7$ along a very poor approximation at $D = 2$ in Fig. 4.17.

4.3.3 Navier-Stokes (stationary)

We construct the corresponding MPS models to our new DQC architecture, and train them identically in two stages. For 10 initial parametrizations, we have not been able to find as many good solutions as with our DQCs at maximal bond dimension $D = 8$. Like in the previous case, we have not been able to find a single good truncation that meets our performance standards. We show the best-found solution along the overall best-found truncation, which is at $D = 7$, in Fig. 4.18.

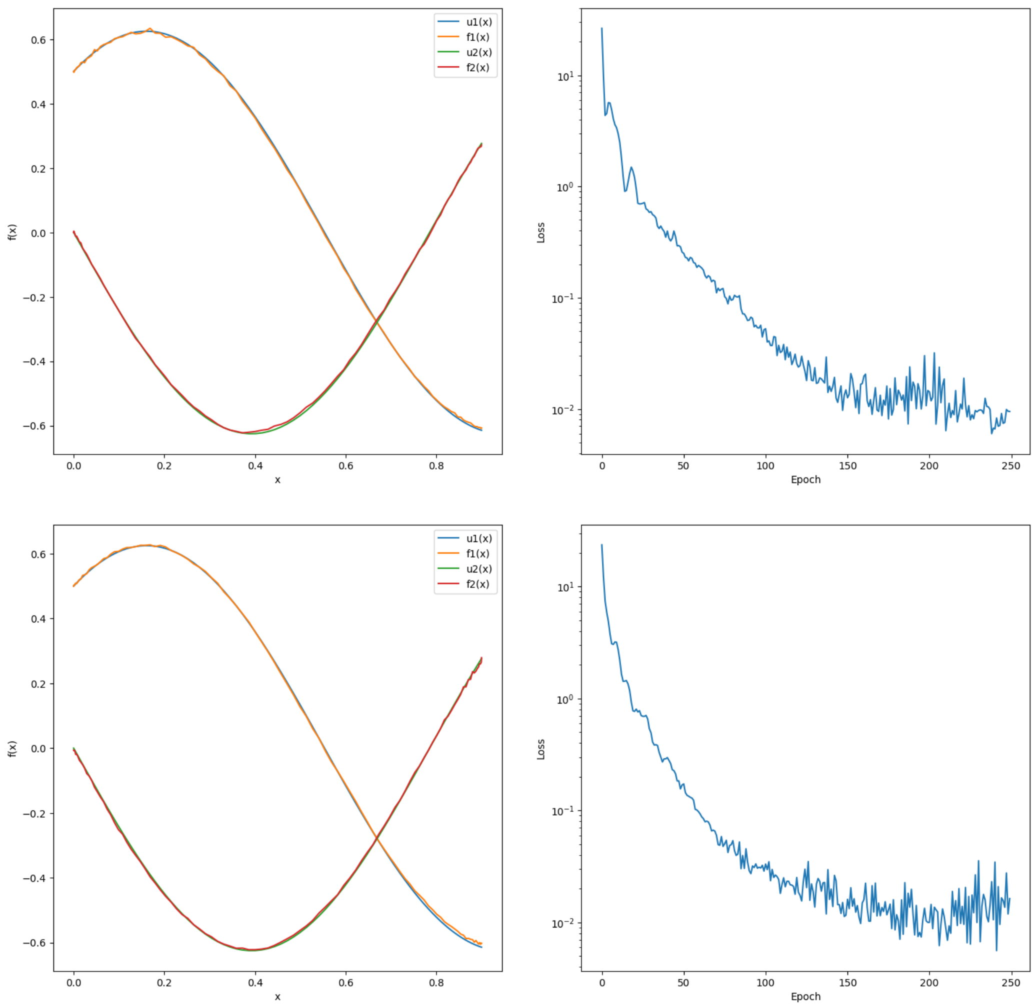

Figure 4.16: *Best-found solution for the* ***Strongly Coupled Oscillators*** *with $\lambda_1 = 5$ against a relatively close solution with matching quality for maximal bond dimension $D = 8$. The best-found solution has $L_f = 0.009484$ and $L_q^1 = 0.000023$, $L_q^2 = 0.000022$. The relatively close solution has $L_f = 0.016298$ and $L_q^1 = 0.000037$, $L_q^2 = 0.000027$. The differential loss L_f does not quite meet our performance standards.*

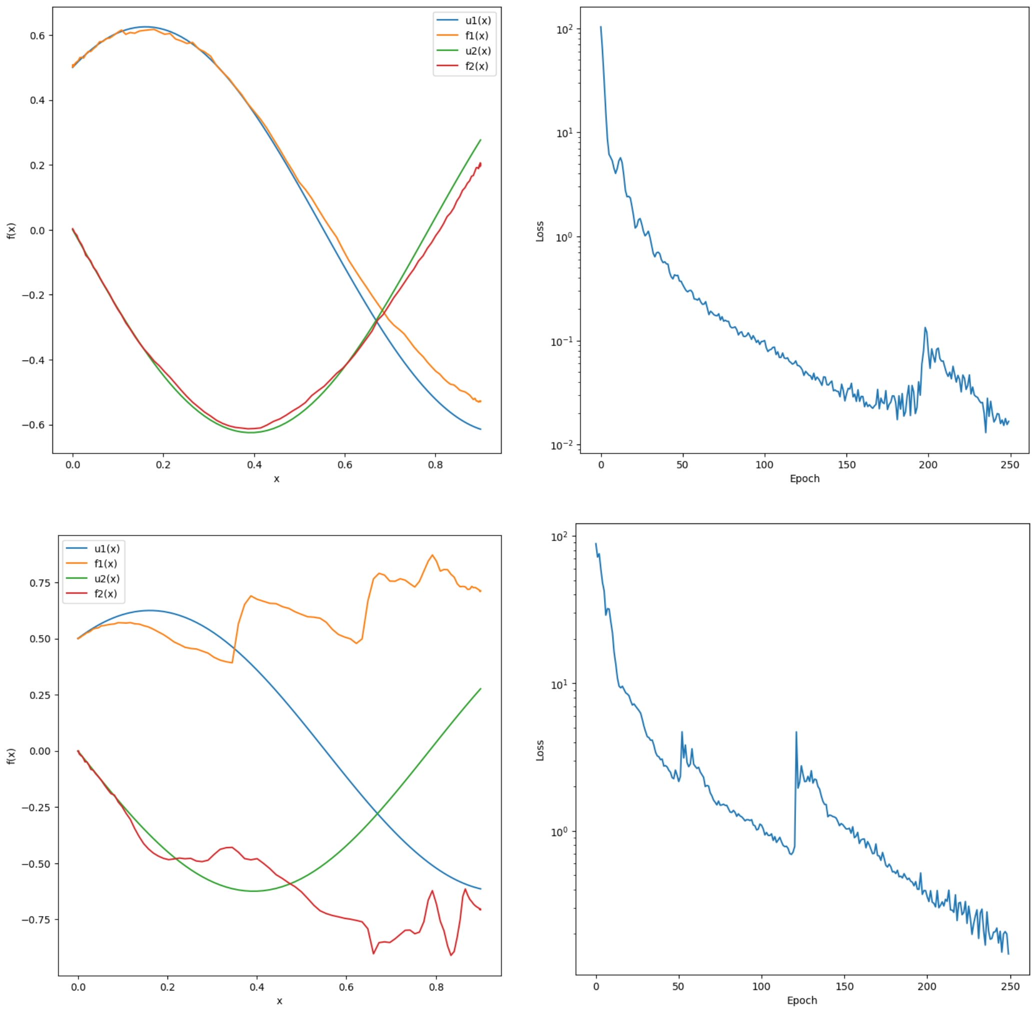

Figure 4.17: *Best-found truncation for the* ***Strongly Coupled Oscillators*** *with $\lambda_1 = 5$ at $D = 7$ along a very poor one at $D = 2$. For $D = 7$, the solution has $L_f = 0.016714$ and $L_q^1 = 0.002716$, $L_q^2 = 0.001278$ which does not meet any of our performance requirements. Similarly for $D = 2$, the model fails in representing the coupled target functions with low entanglement.*

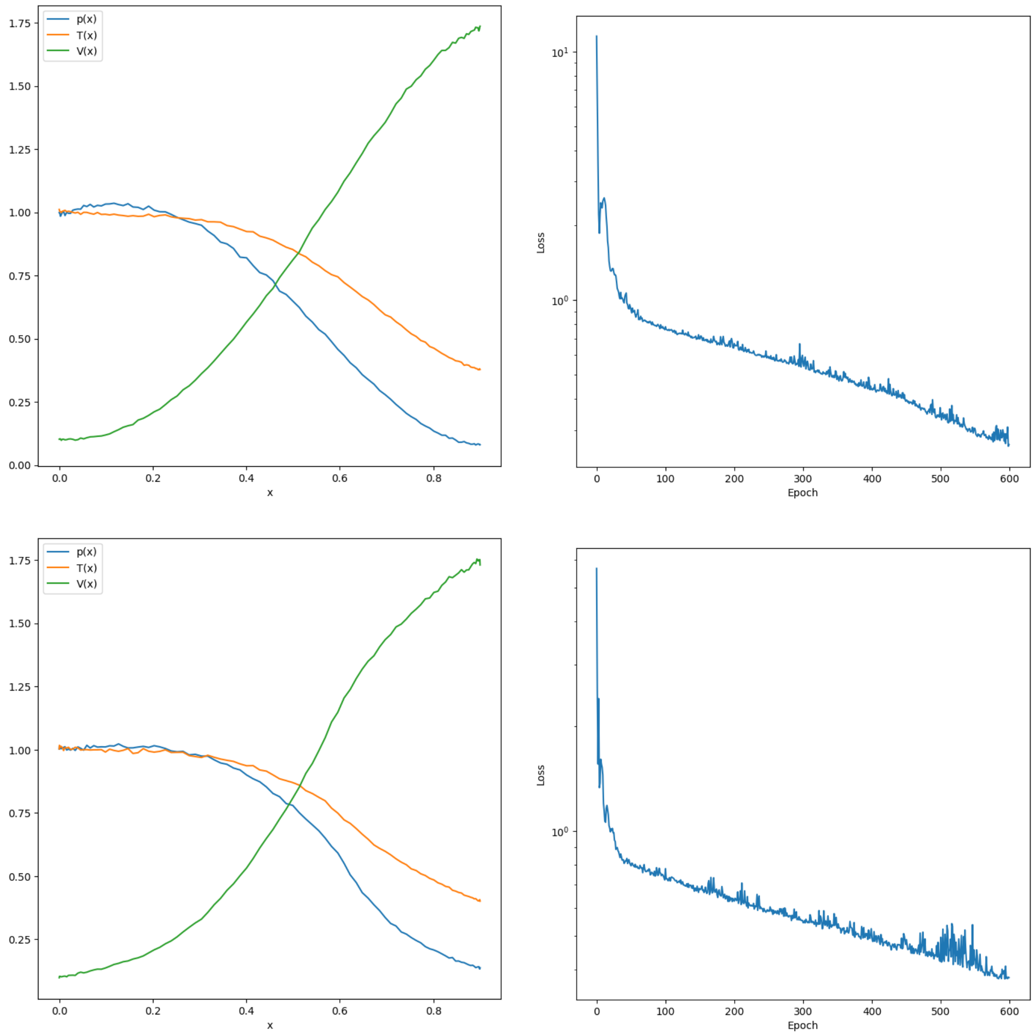

Figure 4.18: *Best-found solution for the stationary* ***Navier-Stokes*** *with maximal bond dimension $D = 8$ against the best-found truncation at $D = 7$, over 10 initial parametrizations. The best-found solution has $L_f = 0.378902$ and matches the expected long-time behavior of the system. The best-found truncation has $L_f = 0.262141$ and still matches the expected long-time behavior of the system but less smoothly.*

Chapter 5

Discussion

5.1 RQ1: How effective are DQCs in terms of expressivity and learnability?

Damped Oscillator We successfully reproduced the results of A. Paine et al. for $\lambda = 8$ over 50 different initial parametrizations. Among these, only 1 out of 50 runs converged to a good solution using equidistant training points (i.e., a uniform grid), while 6 out of 50 succeeded using Chebyshev nodes (i.e., a Chebyshev grid), according to our performance metrics. By relaxing the original performance threshold from $L_q < 10^{-4}$ to $L_q < 10^{-2}$, the number of successful runs increased to 24/50 and 45/50 for the uniform and Chebyshev grids, respectively. This confirms a general trend observed throughout our study: training on Chebyshev over Uniform grids significantly improves the likelihood of convergence toward near-optimal solutions.

In related work, A. Paine suggested that Chebyshev nodes may be better suited for models that encode features using a Chebyshev basis [28]. This can be attributed to the fact that Chebyshev nodes—roots of Chebyshev polynomials of the first kind—are known to suppress Runge's phenomenon [39], i.e., the spurious oscillations that arise near the boundaries of interpolation intervals such as $[-1, 1]$ or any affine rescaling $[a, b]$. These nodes are optimal for minimizing interpolation errors in the L_∞ norm (i.e., maximum pointwise error), particularly for high-degree polynomial approximations. For smooth target functions, this error decays exponentially with the number of nodes—a result supported by both theoretical analysis and empirical evidence [5, 40]. Practically, this advantage

stems from the non-uniform distribution of Chebyshev nodes, which cluster more densely near the domain boundaries—precisely where uniform sampling tends to yield the highest errors. This non-uniformity helps distribute approximation error more evenly, leading to improved numerical performance.
For $\lambda = 20$, we were unable to obtain a single good solution—even with the Chebyshev grid and after relaxing the performance threshold to $< 10^{-2}$. We explored a wide range of DQC configurations, including increased qubit counts, circuit depths, varied learning rates, and extended training durations. A likely reason for this failure lies in the presence of $\tan(\lambda x)$, which introduces an increasing number of singularities as λ grows. These singularities cause numerical instability during training and optimization. This hypothesis is supported by the stiffness index plot in Fig. 4.3, where the most prominent stiffness peak at $x \approx 0.8$ corresponds to the region where DQCs struggled most for $\lambda = 8$. For $\lambda = 20$, the increased frequency and intensity of such stiff regions likely exceeded the capacity of our models to generalize or adapt effectively. This stands in contrast to the results reported by A. Paine et al., who observed successful convergence under comparable conditions, suggesting that additional factors—–such as implicit hyperparameter tuning or initialization sensitivity—–may have contributed to their outcomes.
Overall, our results show a strong dependence on initial parametrization in determining whether a DQC converges to a good solution. While this sensitivity appears to be mitigated when using Chebyshev grids, it remains a limiting factor in very stiff regimes. These cases may inherently require better initialization strategies, stronger regularization, or more expressive ansätze to overcome optimization difficulties.
Finally, we validated our observations by solving a simplified version of the Damped Oscillator, one that excludes problematic singular terms. In this case, we achieved 10/10 successful runs for $\lambda = 8$, and 5/10 for $\lambda = 20$, demonstrating that in the absence of singularities, the system becomes significantly more tractable—even for higher values of λ.

Strongly Coupled Oscillators We successfully reproduced the results of A. Paine et al. for the parameter setting $\lambda_1 = 5$, $\lambda_2 = 3$, obtaining 7 out of 10 good solutions according to our performance metrics. However, for higher values of λ_1, we were unable to achieve a single satisfactory solution, even when the quality loss L_q fell below the 10^{-2} threshold. We conducted an extensive series of experiments with various DQC configurations, including increased qubit counts, deeper circuit depths, and modified learning rates, along with extended training durations. Despite these adjustments, performance did not improve significantly. This degradation appears to stem from the increased influence of the second function u_2 on the first differential equation du_1/dx increases—eventually surpassing the contribution of u_1 itself. Interestingly, this shift seemed to aid the learning of $u1$ more than expected, perhaps due to the way entanglement or parameter updates capture interdependencies in the system. A more detailed investigation is warranted to better understand this effect and to identify optimal parametrization strategies for such strongly coupled regimes.

Navier-Stokes (stationary) We were unable to fully reproduce the results reported by A. Paine et al. for the fluid dynamics problem across 10 different initial parametrizations. Among these, five solutions yielded $L_f \approx 10^{-1}$, but none fell strictly below the 10^{-1} threshold. Without a reference solution, it remains difficult to rigorously assess the quantitative accuracy of the predictions. We contacted the original authors to request additional specifications or reference data, but were unable to obtain them, further limiting our ability to validate the results. Qualitatively, however, the expected dynamics were observed: airflow velocity increased in the convergent region, while both temperature and density decreased. This suggests that the model captured the general behavior, though a sufficiently accurate parametrization was likely not found.
Regarding the discrepancy between the Chebyshev Tower feature map described in the original paper and its implementation in `Qadence`, we note that the resulting Chebyshev expansion is effectively reduced in expressivity. When j is even, the expansion is truncated to a polynomial of degree $j/2$; when j is odd, the expansion includes non-polynomial terms such as square roots and inverse trigonometric functions. This shift prevents the model from reliably spanning polynomial function spaces. The detailed derivations for both cases are provided in Appendix A.3.
In the case of the Navier–Stokes system, DQCs failed to reproduce the steady-state solution when using the Chebyshev Tower feature map, despite producing smooth loss curves and showing no evidence of vanishing gradients or instability. This suggests that the model converged to an inadequate function subspace—most likely due to a mismatch in expressivity introduced by the altered basis. While poor initialization or insufficient regularization may play a role, the core issue appears to be the model's inability to align its learned representations with the structure of the target solution space.

5.2 RQ2: To what extent can MPS-based simulation recover DQC results with maximal bond dimension?

We demonstrated that the convergence behaviors of DQCs and MPS are independent and governed by their own sensitivities to initial parametrization. However, this distinction does not prevent them from achieving similarly good solutions. In one case, both models successfully converged from nearly identical starting points, although the optimizer followed different trajectories in each case. This divergence stems from their fundamentally different architectures MPS is a structured, local representation using tensor networks, whereas DQCs operate with dense unitaries that explore the Hilbert space more freely. As a result, the two models navigate distinct parameter landscapes.
The observed similarity in outcomes may be explained by the fact that both MPS and DQC circuits were constructed from the same gate sequence and observable, and with sufficient entanglement. The numerical divergence likely arises from implementation-specific behaviors within the MPS pipeline—such as the truncation of singular values during SVD decompositions, or precision limitations during tensor contractions used for computing expectation values. In contrast, DQCs rely more heavily on direct matrix multiplications. These differences suggest that TensorCircuit's MPS implementation offers a faithful and efficient simulation of DQC behavior, while still exhibiting slight numerical drift due to inherent representational differences.
Another contributing factor is the use of the `ADAM` optimizer, which adapts learning rates based on the first and second moments of the gradients. This approach helps mitigate gradient noise and stabilize convergence, particularly in the presence of instabilities observed during MPS backpropagation. Our results clearly indicate that this optimization strategy supports the convergence of both models, even in the presence of transient loss spikes.

5.3 RQ3: What is the minimum bond dimension required to accurately capture the solution of a given differential equation?

Damped Oscillator We were able to obtain good approximations of the modified Damped Oscillator using MPS with maximal bond dimension. Across a range of $\lambda = [2\pi, 8, 10, 12, 14]$, the number of successful solutions among 10 initializations gradually decreased from 6 down to 2. It is important to note that MPS and DQCs are fundamentally distinct representations; a good initial parametrization for a DQC does not necessarily translate into a good parametrization for its MPS counterpart. For instance, at $\lambda = 8$, we achieved 6 out of 10 good solutions using DQCs that converged to satisfactory approximations of the target function, as defined by our performance metrics. Yet, for the original Damped Oscillator, we were unable to obtain a single good solution at $\lambda = 8$. As with DQCs, MPS performance is sensitive to initialization—–particularly to the specific parametrized state and the bond dimension. From our results, MPS showed even greater sensitivity, with certain initializations either producing high-quality approximations or failing entirely. However, this observation cannot yet be generalized without testing over a broader range of initial conditions.
Another key observation was that the MPS training process exhibited significantly greater oscillations in the loss curves compared to DQCs. This is likely due to the use of `float32` precision required by TensorCircuit's internal MPS operations. With higher precision (e.g., `float64`), gradient rounding errors would be reduced, potentially leading to more stable optimization and smoother convergence. At larger values of $\lambda > 14$, we encountered *NaN* values in the gradients early in training—presumably caused by exploding gradients resulting from limited numerical precision. Addressing this would require enabling `float64` for internal MPS computations, which may not be currently feasible or guaranteed to resolve the issue.
We also explored MPS truncation by systematically reducing the bond dimension from $D = 2^{N/2}$ down to $D = 1$. Surprisingly, we obtained high-quality approximations at lower bond dimensions in several cases. For example, at $\lambda = 14$, we achieved comparable performance in both L_f and L_q losses at $D = 4$ relative to the maximal bond dimension $D = 8$. In some cases, truncated solutions even outperformed those at maximal entanglement—as seen for $\lambda = 2\pi$, where the best result was achieved at $D = 2$. Nevertheless, a clear trend emerged: as λ increased, the number of good

truncated solutions generally declined. At $\lambda = 2\pi$, four good solutions were found at $D = 2$, but only one was found at the same bond dimension for $\lambda = 8$. For $\lambda = 12$, a single good solution was achieved at $D = 6$, and for $\lambda = 14$, at $D = 4$. In contrast, for the more challenging original Damped Oscillator, no good truncated solution was found.
In all experiments, completely removing entanglement (i.e., using $D = 1$) consistently resulted in poor approximations. For $\lambda = 2\pi$, the MPS still managed to capture some curvature of the nonlinear target function. However, for $\lambda = 14$, the MPS predicted nearly flat boundary behavior, failing to capture any nonlinearity. We therefore conclude that a certain amount of entanglement is necessary for modeling relatively stiff systems with nonlinear solution profiles. The higher the nonlinearity, the greater the entanglement required. While entanglement-constrained approximations often achieved results comparable to their fully entangled counterparts, highly stiff systems featuring sharp transitions or singularities demand significantly more entanglement to be represented accurately.

Strongly Coupled Oscillators We were able to obtain good approximations of the Strongly Coupled Oscillators using MPS with maximal bond dimension. However, across 10 different initializations, none of the truncated models met our performance criteria. Notably, at bond dimension $D = 7$, we encountered the case where both target functions were well-approximated according to the L_q metric, while the L_f value remained slightly above 10^{-1}—-still reasonably close, though not fully satisfactory. As the bond dimension decreased further, the quality of the solution gradually deteriorated, with the degradation appearing first in the L_f metric. This behavior suggests that limiting entanglement impairs the model's ability to capture the coupled derivatives, ultimately leading to a simultaneous decline in the accuracy of the target functions. We therefore conclude that a high degree of entanglement is essential for effectively solving coupled systems.

Navier-Stokes (stationary) We have not been able to obtain a solution matching the quality of the DQC model with maximal bond dimension. While the overall performance of the MPS approximation is not drastically different, it remains significantly worse—by approximately one order of magnitude—than the solution reported by A. Paine et al. Across 10 different initializations, none yielded a satisfactory truncation according to our performance metrics. At bond dimension $D = 7$, the solution already begins to lose granularity. Nonetheless, the long-time behavior is still qualitatively preserved: the nonlinear dynamics are correctly captured, with density, temperature, and velocity converging at $x = 0.5$, followed by a divergence of velocity in the supersonic regime. We therefore conclude that a nontrivial amount of entanglement is necessary to faithfully represent complex nonlinear contributions in multi-equation systems. In particular, for convergent-divergent functions, insufficient entanglement leads to a clear degradation in solution quality, manifesting as reduced smoothness, instability, and spurious oscillations—especially when evaluated over finely discretized domains.

Chapter 6

Conclusions

This thesis investigated the role of entanglement in the efficacy of Differentiable Quantum Circuits (DQCs) for solving differential equations (DEs). By systematically analyzing the entanglement requirements for accurate function approximation, we established a direct connection between entanglement and expressivity-the capacity to approximate any function belonging to a particular function space defined in a prescribed domain up to arbitrary precision with respect to a specific distance-of variational quantum models in scientific computing [26].
We evaluated the expressivity and learnability of DQCs on three representative DEs: a damped oscillator, a strongly coupled system of oscillators, and a stationary Navier-Stokes system. We considered generalization capability of DQCs as given in A. Paine et al.'s work [18], under increasing nonlinearity, stiffness, and coupling strength via the relevant control parameters. For the **Damped Oscillator**, we observed high sensitivity on the initial parametrization of variational angles. This dependence was mitigated when training on Chebyshev grids, likely due to their improved numerical stability and interpolation properties in polynomial approximation theory [40]. However, increasing the stiffness via large λ values introduced singularities (via $tan(\lambda x)$) and steep gradients, which significantly degraded the model's convergence and accuracy. As a workaround, we introduced a non-singular version of the oscillator that retained stiffness properties without introducing non-physical divergences. In the case of **Strongly Coupled Oscillators**, we reproduced A. Paine et al's results for moderate coupling strength ($\lambda_1 = 5$) but failed to find good solutions at higher couplings. This suggests that DQCs with shallow circuits and fixed

architecture may lack the expressive capacity to capture strongly correlated, nonlinear behaviors. For the **Navier-Stokes** equations, we successfully reproduced steady-state approximations consistent with those in the original study. This case confirmed the ability of DQCs to encode nonlinear relationships across equations, and highlights a promising route towards applying DQCs to more complex fluid systems.

We demonstrated numerically that MPS models with maximal bond dimension can produce DQCs predictions up to numerical precision. This equivalence holds at the level of function approximation, but the two models, owing to their fundamentally different parameterizations and optimization landscapes, often follow distinct convergence trajectories. Consequently, equivalence in expressivity does not imply equivalence in convergence dynamics, particularly under identical initializations. Nonetheless, for all three examples, MPS consistently matched DQC performance over 10 independent initializations, reinforcing their utility as classical benchmarks for assessing quantum circuit expressivity.

To characterize the entanglement necessary to solve a given DE, we systematically reduced the MPS bond dimension and tracked solution quality. Results varied significantly by problem class. For the new **Damped Oscillator**, we found that low bond dimensions (often half the DQC's entanglement capacity) were sufficient for approximating solutions in low-to-moderate stiffness regimes. However, for the original stiff oscillator, no satisfactory solution was found under truncation, reinforcing the hypothesis that high stiffness demands high entanglement. For the **Strongly Coupled Oscillators**, no good approximations were achieved under bond dimension truncation, regardless of initialization. The gradual degradation of solution quality as bond dimension decreased supports the conclusion that entanglement is critical for solving strongly coupled systems. For the Navier-Stokes equations, all truncations failed to capture the system's nonlinear structure, even though the problem is defined in a 1D domain. This result indicates that even low-dimensional systems can have intrinsically high entanglement requirements when nonlinearity and cross-variable interactions dominate.

Despite the valuable insights gained, several limitations must be acknowledged. We limited our analysis to Chebyshev-based feature maps and a hardware-efficient ansatz (HEA) of shallow depth. This choice was made to isolate entanglement effects and avoid confounding factors from excess expressivity. However, other ansatz types—such as layered hardware-efficient ansätze with long-range entanglement [7], or problem-inspired ansätze like QAOA [10]-could lead to different entanglement scaling and improved robustness to stiffness. In such cases, the cost Hamiltonian could

be defined to penalize residuals at collocation points derived from Chebyshev grids, while the mixer may be designed to encourage smooth transitions between adjacent degrees in the feature map, preserving the structure induced by the spectral encoding. Solution fidelity was assessed using relatively strict performance thresholds ($L_f < 10^{-2}$, $L_q < 10^{-4}$), which may have excluded valid approximations that qualitatively capture main features of the solution. Relaxing these metrics could provide a broader perspective on the usefulness of near-optimal solutions in noisy or chaotic regimes. Only 10 initializations were used per experiment. While this is standard in variational quantum model evaluations, it limits our ability to fully explore the solution landscape-particularly for highly rugged loss surfaces. Increasing the number of trials could improve the likelihood of finding good truncations. Moreover, the use of automatic differentiation (autodiff) rather than parameter shift rules (PSR) facilitated large-scale training but introduces potential artifacts in gradient flow. While analytic equivalence holds in theory, the computational graphs used in autodiff may exhibit hardware- or library-specific limitations [4].
Building on the outcomes and limitations of this study, we outline three promising avenues for future research. First, extending this framework to dynamical, chaotic PDEs such as the full Navier–Stokes system (in 2D or 3D) would probe whether entanglement continues to act as a bottleneck in higher-dimensional, time-evolving problems. Second, future models could employ variational circuits with dynamically allocated entanglement, adapting bond dimensions or entangling depth to local problem complexity. Hierarchical tensor networks (e.g., Tree Tensor Networks or MERA) and multi-scale quantum circuits may offer more resource-efficient models for problems with localized stiffness or multiscale structure [9]. Third, this study suggests that differential equations can be characterized by their entanglement demand-a measure of the quantum correlations required to accurately approximate their solutions using variational quantum models. Our results indicate that structural properties such as stiffness, nonlinearity, and coupling strength significantly affect this demand. Formalizing this concept could serve as a guiding principle for constructing or identifying differential equations that are particularly well-suited for quantum solvers. Rather than treating quantum advantage as an all-or-nothing threshold, this perspective emphasizes designing or selecting DEs whose structure and solution properties align with the strengths of entanglement-enabled models, thereby maximizing the potential benefit of quantum resources in scientific computing.

In summary, this thesis demonstrates that the expressivity of variational quantum models—defined as their ability to approximate solutions to differential equations within a specified function space and accuracy threshold—is directly constrained by the level of entanglement in the circuit. Through systematic experiments across representative systems, we showed that specific structural properties of the differential equations—such as stiffness, nonlinearity, and coupling—induce solution functions with features that demand increasing entanglement to be accurately captured. These results establish entanglement as a necessary computational resource not in relation to the equation alone, but in relation to the complexity of the solution it generates. To our knowledge, this is among the first empirical studies to explicitly quantify the link between equation structure, solution behavior, and entanglement requirements in the context of variational quantum solvers. This understanding may inform the development of practical heuristics for entanglement allocation, model architecture design, and training strategies-and, more fundamentally, provide a basis for identifying or constructing differential equations that would benefit most from quantum computing methods.

Reflections on Impact and Interdisciplinary Positioning

This thesis lies at the intersection of machine learning and quantum computing, grounded in the shared goal of function approximation for scientific tasks—in particular, solving differential equations (DEs). Rather than combining disparate methods, its interdisciplinary stems from how it formulates its research questions across both fields. On one side, it considers models like Differentiable Quantum Circuits (DQCs), where entanglement is both a structural constraint and a computational resource. On the other, it evaluates classical simulations via Matrix Product States (MPS), which represent low-entanglement quantum states efficiently via tensor network structures with tunable bond dimension. Both models are examined not merely as simulation tools but as parameterized function approximators, optimized to minimize the residual of a target DE. This framing situates the study within a broader theoretical effort to characterize and compare expressive function spaces under structural constraints across classical and quantum paradigms.

The thesis addresses this intersection through three central research questions. **RQ1** concerns the expressivity of DQCs in approximating target functions governed by nonlinear, stiff, or coupled DEs. It investigates how the entanglement encoded in the variational ansatz relates to the model's ability to approximate any function within a prescribed functional class, up to arbitrary precision with respect to a chosen distance. **RQ2** extends this expressivity analysis by comparing the function-approximating capabilities of structurally distinct models—DQCs and MPS—with equivalent entanglement capacity, asking whether entanglement alone determines approximation performance, independent of the model's architecture. **RQ3** explores the implications of restricting entanglement via Schmidt truncation, assessing the degradation in function approximation quality when using reduced MPS representations to emulate full-entanglement DQC states. These questions are articulated from a function space perspective, focusing on the coverage, limitations, and structure of the sets of functions each model can represent over the input domain.

This contributes to ongoing discussions in quantum machine learning regarding the scalability and functional relevance of entanglement. By framing entanglement as a quantitative constraint on expressivity rather than as a binary computational resource, the thesis clarifies when quantum correlations become indispensable for capturing key features of complex physical models. In parallel, it offers the applied mathematics and engineering communities a new viewpoint on DE solving—one that is function-oriented rather than discretization-based, emphasizing the ability to capture global solution structures over fine-grained local updates.

More broadly, the work reflects on the implications of entanglement-aware modeling for hybrid classical-quantum pipelines. Understanding the role of entanglement in shaping representational capacity can inform resource allocation in quantum hardware, architecture selection in variational quantum algorithms, and even the design of curricula in quantum engineering programs. While the conclusions drawn are specific to DQCs and MPS-based simulators, the methodology developed herein—grounded in function space analysis and entanglement-constrained approximation—offers a transferable framework for assessing the trade-offs between classical and quantum modeling across scientific disciplines.

Code

Code to reproduce the figures and explore further settings can be found in the following GitHub repository: `https://github.com/Roman20022002/MEP.git`.

Acknowledgements

I would like to thank Vedran Dunjko for his supervision, guidance, and enthusiasm throughout this project. His support helped shape the direction of this research and provided an environment where curiosity and creativity were encouraged. I am particularly grateful to Riccardo Molteni for his constant support, detailed feedback, and close involvement on a daily basis. His patience, technical insight, and readiness to help at every stage of the work were invaluable, and this thesis would not have taken its final form without his dedicated mentorship.
I also want to warmly thank Annie Paine for taking the time to meet with us and for generously exchanging ideas on Differentiable Quantum Circuits. Her openness and willingness to engage in technical discussions added both clarity and motivation to this project.
The experimental results in this thesis relied heavily on the computational power provided by the ALICE High Performance Computing cluster at Leiden University. I am very thankful for this essential resource, and I would like to extend special thanks to the system administrators for their reliable and responsive support throughout the research process.
Finally, I sincerely thank Matthias Möller and Evert van Nieuwenburg for their time, constructive feedback, and contributions as second readers of this thesis. Their careful evaluation and thoughtful suggestions helped improve both the structure and clarity of this work.

Bibliography

[1] Eric R. Anschuetz and Bobak T. Kiani. Beyond barren plateaus: Quantum variational algorithms are swamped with traps. *arXiv preprint arXiv:2205.05786*, 2022.

[2] Andrew Arrasmith, M. Cerezo, Piotr Czarnik, Lukasz Cincio, and Patrick J. Coles. Effect of barren plateaus on gradient-free optimization. *arXiv preprint arXiv:2011.12245*, 2021.

[3] Marco Ballarin, Stefano Mangini, Simone Montangero, Chiara Macchiavello, and Riccardo Mengoni. Entanglement entropy production in quantum neural networks. *arXiv preprint arXiv:2206.02474*, 2022. Last revised 24 May 2023.

[4] Marcello Benedetti, Delfina Garcia-Pintos, Oscar Perdomo, Vicente Leyton-Ortega, Yunseong Nam, and Alejandro Perdomo-Ortiz. A generative modeling approach for benchmarking and training shallow quantum circuits. *arXiv preprint arXiv:1801.07686*, 2018. `https://arxiv.org/abs/1801.07686`.

[5] John P. Boyd. *Chebyshev and Fourier Spectral Methods*. Dover Publications, 2nd edition, 2001.

[6] John C. Butcher. *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons, 2nd edition, 2008.

[7] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *arXiv preprint arXiv:2012.09265*, 2020. `https://arxiv.org/pdf/2012.09265`.

[8] Kuan-Cheng Chen, Chen-Yu Liu, Yu Shang, Felix Burt, and Kin K. Leung. Distributed quantum neural networks on distributed photonic quantum computing. *arXiv preprint*, May 2025. Last revised May 13, 2025.

[9] Glen Evenbly and Guifré Vidal. Tensor network states and geometry. *Journal of Statistical Physics*, 145(4):891–918, 2011. `https://arxiv.org/abs/1106.1082`.

[10] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[11] Juan José García-Ripoll. Quantum-inspired algorithms for multivariate analysis: from interpolation to partial differential equations. *Quantum*, 5:431, 2021.

[12] C. W. Gear. *Numerical Initial Value Problems in Ordinary Differential Equations*. Prentice-Hall, 1971.

[13] Edward Grant, Leonard Wossnig, Mateusz Ostaszewski, and Marcello Benedetti. An initialization strategy for addressing barren plateaus in parametrized quantum circuits. *Quantum*, 3:214, 2019.

[14] Ernst Hairer and Gerhard Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, volume 14 of *Springer Series in Computational Mathematics*. Springer, 2nd edition, 1996.

[15] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15):150502, 2009.

[16] Matthew B Hastings. An area law for one-dimensional quantum systems. *Journal of Statistical Mechanics: Theory and Experiment*, 2007(08):P08024, 2007.

[17] Oleksandr Kyriienko and Vincent E. Elfving. Generalized quantum circuit differentiation rules. *Physical Review A*, 104(5):052417, 2021.

[18] Oleksandr Kyriienko, Annie E. Paine, and Vincent E. Elfving. Solving nonlinear differential equations with differentiable quantum circuits. *arXiv preprint arXiv:2011.10395*, 2020.

[19] Oleksandr Kyriienko, Annie E Paine, and Vincent E Elfving. Protocols for trainable and differentiable quantum generative modelling. *arXiv preprint arXiv:2202.08253*, 2022.

[20] Martin Larocca, Supanut Thanasilp, Samson Wang, Kunal Sharma, Jacob Biamonte, Patrick J. Coles, Lukasz Cincio, Jarrod R. McClean, Zoë Holmes, and M. Cerezo. Barren plateaus in variational quantum computing. *arXiv preprint arXiv:2405.00781*, 2024.

[21] Wim Lavrijsen, Ana Tudor, Juliane Müller, Costin Iancu, and Wibe de Jong. Classical optimizers for noisy intermediate-scale quantum devices. *arXiv preprint arXiv:2004.03004*, 2020.

[22] Leiden University. ALICE: Leiden Computer Cluster. `https://www.universiteitleiden.nl/en/research/research-facilities/alice-leiden-computer-cluster`, n.d. Accessed: 2025-06-03.

[23] Lorenzo Leone, Salvatore F. E. Oliviero, Lukasz Cincio, and M. Cerezo. On the practical usefulness of the hardware efficient ansatz. *arXiv preprint arXiv:2211.01477*, 2022. Last revised 26 Jun 2024.

[24] Lorenzo Leone, Salvatore F. E. Oliviero, Lukasz Cincio, and M. Cerezo. On the practical usefulness of the hardware efficient ansatz. *arXiv preprint arXiv:2211.01477*, 2024. Version 2, last revised 26 Jun 2024.

[25] Edward N. Lorenz. Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2):130–141, 1963.

[26] Alberto Manzano, David Dechant, Jordi Tura, and Vedran Dunjko. Approximation and generalization capacities of parametrized quantum circuits for functions in sobolev spaces. *arXiv preprint arXiv:2307.14792*, 2023. Version 3, revised March 2025.

[27] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition edition, 2010.

[28] Annie E. Paine. *Quantum Machine Learning and Quantum Protocols for Solving Differential Equations*. Ph.d. dissertation, University of Exeter, 2024.

[29] Annie E Paine, Vincent E Elfving, and Oleksandr Kyriienko. Quantum quantile mechanics: Solving stochastic differential equations for generating time-series. *arXiv preprint arXiv:2108.03190*, 2021.

[30] Annie E Paine, Vincent E Elfving, and Oleksandr Kyriienko. Physics-informed quantum machine learning: Solving nonlinear differential equations in latent spaces without costly grid evaluations. *arXiv preprint arXiv:2308.01827*, 2023.

[31] Alfio Quarteroni, Riccardo Sacco, and Fausto Saleri. *Numerical Mathematics*. Springer Science & Business Media, 2nd edition, 2007.

[32] Michael Ragone, Bojko N. Bakalov, Frédéric Sauvage, Alexander F. Kemper, Carlos Ortiz Marrero, Martin Larocca, and M. Cerezo. A lie algebraic theory of barren plateaus for deep parameterized quantum circuits. *arXiv preprint arXiv:2309.09342*, 2023.

[33] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[34] Ulrich Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011.

[35] Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99(3):032331, 2019.

[36] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. The effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103(3):032430, 2021.

[37] Dominik Seitz, Niklas Heim, João P. Moutinho, Roland Guichard, Vytautas Abramavicius, Aleksander Wennersteen, Gert-Jan Both, Anton Quelle, Caroline de Groot, Gergana V. Velikova, Vincent E. Elfving, and Mario Dagrada. Qadence: a differentiable interface for digital-analog programs. *arXiv preprint arXiv:2401.09915*, 2024.

[38] Lloyd N. Trefethen. *Spectral Methods in MATLAB*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2000.

[39] Lloyd N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2013.

[40] Lloyd N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2019.

[41] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. SIAM, 1997.

[42] Guifre Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical Review Letters*, 91(14):147902, 2003.

[43] Guifré Vidal. Efficient simulation of one-dimensional quantum many-body systems. *Physical Review Letters*, 93(4):040502, 2004.

[44] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021.

[45] Chelsea A. Williams, Annie E. Paine, Hsin-Yu Wu, Vincent E. Elfving, and Oleksandr Kyriienko. Quantum chebyshev transform: Mapping, embedding, learning and sampling distributions. *arXiv preprint arXiv:2306.17026*, 2023.

[46] Xuchen You and Xiaodi Wu. Exponentially many local minima in quantum neural networks. *arXiv preprint arXiv:2110.06002*, 2021.

[47] Mingjian Zhang, Shihao Zhang, Fu Cong, Zheng Xu, Xiaohui Liu, Xinyu Wu, Cheng Guo, Haixuan Yao, Shenggen Lu, Dong-Ling Deng, et al. Tensorcircuit: A quantum software framework for the nisq era. *arXiv preprint arXiv:2205.11487*, 2022.

Appendix A

Appendix

A.1 Derivation of the Stiff-Index for the Damped Oscillator

We consider the original **Damped Oscillator** ODE from A. Paine et al.'s work [18]:

$$\frac{du}{dx} = -\lambda u(x)\,(\kappa + \tan(\lambda x))$$

The first derivative is given as:

$$u'(x) = -\lambda u(x)\,(\kappa + \tan(\lambda x))$$

Differentiating both sides, we obtain:

$$u''(x) = \frac{d}{dx}\left[-\lambda u(x)\,(\kappa + \tan(\lambda x))\right] = -\lambda u'(x)\,(\kappa + \tan(\lambda x)) - \lambda u(x) \cdot \frac{d}{dx}[\tan(\lambda x)]$$

Using $\frac{d}{dx}[\tan(\lambda x)] = \lambda \sec^2(\lambda x)$, we get:

$$u''(x) = -\lambda u'(x)\,(\kappa + \tan(\lambda x)) - \lambda^2 u(x) \sec^2(\lambda x)$$

We use as a stiffness indicator the following stiff-index defined by C.W. Gear [12]:

$$S(x) = \left|\frac{u''(x)}{u'(x)}\right|$$

Substituting, we obtain:

$$S(x) = \left|\frac{-\lambda u'(x)(\kappa + \tan(\lambda x)) - \lambda^2 u(x) \sec^2(\lambda x)}{-\lambda u(x)(\kappa + \tan(\lambda x))}\right|$$

Splitting terms and simplifying, we get the final expression of the stiff-index for our problem that depends on both k and λ:

$$S(x) = \left|\lambda(\kappa + \tan(\lambda x)) + \frac{\lambda^2 \sec^2(\lambda x)}{\lambda(\kappa + \tan(\lambda x))}\right| = \left|-\lambda(\kappa + \tan(\lambda x)) + \frac{\lambda \sec^2(\lambda x)}{\kappa + \tan(\lambda x)}\right|$$

A.2 Analytical solution of the Strongly Coupled Oscillators

The **Strongly Coupled Oscillators** is a system of coupled ODEs defined as:

$$\begin{cases} \frac{du_1}{dx} = 5u_2 + 3u_1 \\ \frac{du_2}{dx} = -3u_2 - 5u_1 \end{cases} \quad \text{with initial conditions: } u_1(0) = 0.5, u_2(0) = 0$$

We define the vector function:

$$\mathbf{u}(x) = \begin{bmatrix} u_1(x) \\ u_2(x) \end{bmatrix} \quad \Rightarrow \quad \frac{d\mathbf{u}}{dx} = A\mathbf{u}, \quad \text{where } A = \begin{bmatrix} 3 & 5 \\ -5 & -3 \end{bmatrix}$$

To solve this system, we compute the eigenvalues of matrix A by solving the characteristic equation:

$$\det(A - \lambda I) = \det\left(\begin{bmatrix} 3-\lambda & 5 \\ -5 & -3-\lambda \end{bmatrix}\right) = (3-\lambda)(-3-\lambda) + 25 = \lambda^2 + 16 = 0 \Rightarrow \lambda = \pm 4i$$

Now, we find the eigenvector for $\lambda = 4i$. We solve:

$$(A - 4iI)\mathbf{v} = 0 \Rightarrow \begin{bmatrix} 3-4i & 5 \\ -5 & -3-4i \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = 0$$

From the first row:

$$(3-4i)v_1 + 5v_2 = 0 \Rightarrow v_2 = -\frac{3-4i}{5} v_1$$

Let $v_1 = 1$, then:

$$\mathbf{v} = \begin{bmatrix} 1 \\ -\frac{3}{5} + \frac{4}{5}i \end{bmatrix}$$

The general complex solution is therefore:

$$\mathbf{u}(x) = c_1 e^{4ix}\mathbf{v} + c_2 e^{-4ix}\bar{\mathbf{v}}$$

To write a real-valued solution, we express it using real and imaginary parts:

$$\mathbf{u}(x) = a \cdot \Re(e^{4ix}\mathbf{v}) + b \cdot \Im(e^{4ix}\mathbf{v})$$

Now compute:

$$\Re(e^{4ix}\mathbf{v}) = \cos(4x)\begin{bmatrix}1\\ -\frac{3}{5}\end{bmatrix} - \sin(4x)\begin{bmatrix}0\\ \frac{4}{5}\end{bmatrix} = \begin{bmatrix}\cos(4x)\\ -\frac{3}{5}\cos(4x) - \frac{4}{5}\sin(4x)\end{bmatrix}$$

$$\Im(e^{4ix}\mathbf{v}) = \sin(4x)\begin{bmatrix}1\\ -\frac{3}{5}\end{bmatrix} + \cos(4x)\begin{bmatrix}0\\ \frac{4}{5}\end{bmatrix} = \begin{bmatrix}\sin(4x)\\ -\frac{3}{5}\sin(4x) + \frac{4}{5}\cos(4x)\end{bmatrix}$$

Hence, the general real solution becomes:

$$\mathbf{u}(x) = a\begin{bmatrix}\cos(4x)\\ -\frac{3}{5}\cos(4x) - \frac{4}{5}\sin(4x)\end{bmatrix} + b\begin{bmatrix}\sin(4x)\\ -\frac{3}{5}\sin(4x) + \frac{4}{5}\cos(4x)\end{bmatrix}$$

We apply the initial conditions. At $x = 0$:

$$u_1(0) = a = 0.5 \qquad u_2(0) = -\frac{3}{5}a + \frac{4}{5}b = 0 \Rightarrow -\frac{3}{5}\cdot 0.5 + \frac{4}{5}b = 0 \Rightarrow b = \frac{3}{10}\cdot\frac{5}{4} = \frac{3}{8} = 0.375$$

Substitute $a = 0.5$, $b = 0.375$ into the general solution:

$$u_1(x) = 0.5\cos(4x) + 0.375\sin(4x)$$

$$u_2(x) = \left(-\frac{3}{5}\cdot 0.5 + \frac{4}{5}\cdot 0.375\right)\cos(4x) + \left(-\frac{4}{5}\cdot 0.5 - \frac{3}{5}\cdot 0.375\right)\sin(4x) = -0.625\sin(4x)$$

$$\boxed{u_1(x) = 0.5\cos(4x) + 0.375\sin(4x)} \qquad \boxed{u_2(x) = -0.625\sin(4x)}$$

A.3 Chebyshev Polynomial Expansions of the Operator $\hat{R}_{y,j}(\varphi(x))$

We provide in this section the derivations of the rotational gate $\hat{R}_{y,j}(\varphi(x))$ in terms of Chebyshev polynomials, both for the full-angle and half-angle cases. These expressions are useful in understanding the representational power of the **Chebyshev tower feature map**.

Original Full-Angle Derivation (for comparison). We begin with the case where the rotation angle is $j \cdot \arccos(x)$ as defined in the original work by A. Paine et al. [18]. The exponential operator can be expanded as:

$$\hat{R}_{y,j}(\varphi(x)) = \exp\left(-i \cdot j \cdot \arccos(x) \cdot \hat{Y}_j\right)$$

Using the identity:

$$\exp(-i\theta\hat{Y}_j) = \cos(\theta)\,\mathbb{1}_j - i\sin(\theta)\,\hat{Y}_j$$

Substituting $\theta = j \cdot \arccos(x)$, we obtain:

$$\hat{R}_{y,j}(\varphi(x)) = \cos(j\arccos(x))\,\mathbb{1}_j - i\sin(j\arccos(x))\,\hat{Y}_j$$

Then applying Chebyshev identities:

$$\cos(j\arccos(x)) = T_j(x), \qquad \sin(j\arccos(x)) = \sqrt{1-x^2} \cdot U_{j-1}(x)$$

we arrive at:

$$\hat{R}_{y,j}(\varphi(x)) = T_j(x)\,\mathbb{1}_j - i\sqrt{1-x^2} \cdot U_{j-1}(x) \cdot \hat{Y}_j$$

This expression is a degree-j polynomial in x (modulo the $\sqrt{1-x^2}$ factor in the sine term), and represents the most expressive case within the Chebyshev basis.

Half-Angle Derivation. We now consider the case where the rotation angle is $\theta = \frac{j}{2} \cdot \arccos(x)$, which we suspect to be the one implemented in `Qadence` library.

Using the same exponential identity:

$$\hat{R}_{y,j}(\varphi(x)) = \exp\left(-i \cdot \frac{j \cdot \arccos(x)}{2} \cdot \hat{Y}_j\right) = \cos\left(\frac{j \cdot \arccos(x)}{2}\right)\mathbb{1}_j - i\sin\left(\frac{j \cdot \arccos(x)}{2}\right)$$

We consider the two following cases separately. **Case 1:** j **even (**$j = 2k$**)** In this case:

$$\theta = \frac{2k \cdot \arccos(x)}{2} = k \cdot \arccos(x)$$

Using the same Chebyshev relations:

$$\cos(k\arccos(x)) = T_k(x), \qquad \sin(k\arccos(x)) = \sqrt{1-x^2} \cdot U_{k-1}(x)$$

we obtain the half-angle rotation as:

$$\hat{R}_{y,2k}(\varphi(x)) = T_k(x)\,\mathbb{1}_{2k} - i\sqrt{1-x^2}\cdot U_{k-1}(x)\cdot \hat{Y}_{2k}$$

This is again a polynomial expression in x, but only up to degree $k = j/2$. Hence, the expressive capacity of the operator is reduced.

Case 2: j **odd (**$j = 2k+1$**)** This case requires a half-angle identity since the angle $\theta = \frac{(2k+1)\arccos(x)}{2}$ is not a multiple of $\arccos(x)$. Let $\alpha = (2k+1)\cdot\arccos(x)$, so that:

$$\theta = \frac{\alpha}{2}$$

Applying the half-angle formulas:

$$\cos\left(\frac{\alpha}{2}\right) = \sqrt{\frac{1+\cos(\alpha)}{2}}, \qquad \sin\left(\frac{\alpha}{2}\right) = \sqrt{\frac{1-\cos(\alpha)}{2}}$$

and using the identity $\cos(\alpha) = T_{2k+1}(x)$, we obtain:

$$\hat{R}_{y,2k+1}(\varphi(x)) = \sqrt{\frac{1+T_{2k+1}(x)}{2}}\,\mathbb{1}_{2k+1} - i\sqrt{\frac{1-T_{2k+1}(x)}{2}}\cdot \hat{Y}_{2k+1}$$

This expression is not a polynomial in x, due to the square root. Therefore, the operator no longer lies within the span of a finite-degree polynomial basis. This limits its usefulness in contexts that depend on polynomial approximation.

A.4 Summary of Compute Resources Used on ALICE

All experiments for MPS truncation (**RQ3**) have been conducted on the Academic Leiden Interdisciplinary Cluster Environment (ALICE) at Leiden University. In the following table, we provide the computational time and used memory per job, along the HPC Cluster partition on which they have been run. Mind that for each experiment, jobs were repeated for 8 bond dimensions at most, and for 10 initial parametrizations. Hence, the maximum array size of jobs was 80 for a single experiment.

Experiment	**Avg. RAM (GB)**	**Avg. Time (h)**	**Partition**
Damped Oscillator ($\lambda = 2\pi$)	7.01	0.80	gpu-medium
Damped Oscillator ($\lambda = 8$)	7.30	1.16	gpu-medium
Damped Oscillator ($\lambda = 10$)	6.85	1.34	gpu-medium
Damped Oscillator ($\lambda = 12$)	6.87	1.58	gpu-medium
Damped Oscillator ($\lambda = 14$)	7.62	1.56	gpu-medium
Original Damped Oscillator ($\lambda = 8$)	6.92	3.49	gpu-medium
Strongly Coupled Oscillators ($\lambda_1 = 5$)	11.65	3.51	gpu-medium
Navier-Stokes (stationary)	18.9	32.10	gpu-long